ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

LRE Language Reasoning Education

StructPF:

Towards Pedagogically Sound Productive Failure Task Generation with Large Language Models

Master's Thesis

Ouyu Xu
Department of Computer Science (D-INFK)
Learning and Reasoning Lab (LRE)

Supervisors: Dr. Jakub Macina
Prof. Dr. Mrinmaya Sachan

October 2026

Abstract

Productive Failure (PF) is a learning design in which students attempt a challenging problem before they are taught the target concept. A good PF task lets students start from what they already know and try several approaches, but chooses its numbers so that these familiar methods fall short and students see why the target concept is needed. Such tasks are costly to design, and few published examples exist. While a large language model (LLM) could be prompted to write PF tasks in one pass, such prompting mostly produces polished tasks that look like PF on the surface but do not work like it. In our replication of this approach, the generated tasks scored high on the rubric from prior work, yet most of them could be solved with methods students already knew.

To close this gap, we have developed StructPF, a pipeline that follows the design principles of PF. It retrieves the students' prior knowledge from curriculum data, selects task numbers for which a familiar method falls short and verifies them by calculation, and only then writes the task for students. Since prior rubrics only evaluate PF tasks on a surface level, we also developed PF Eval, a benchmark that tests the in-depth structure of a task instead.

Our evaluation showed that familiar methods fall short in four of five StructPF tasks compared with one in ten one-shot tasks, and that curriculum data keeps the generator from assuming that students already know the target concept. StructPF still trails published expert tasks, and simulated students did not reproduce the approaches of real students. These results suggest that PF generation is limited by knowledge about learners rather than by model size. LLMs are trained to solve tasks, not to anticipate how students think before they are taught, so this knowledge has to come from curricula and experts, and StructPF shows one way to bring it in.

Acknowledgements

First and foremost, I would like to thank my supervisor Jakub Macina, who accompanied this thesis from the first proposal to the last figure. Whenever I got stuck, he had a paper to point me to, a resource I did not know existed, or an idea that made me look at the problem from a different side. Many of the directions in this thesis began as a short remark in one of our meetings, and I am grateful for the time and patience he put into them. I would also like to thank Prof. Dr. Mrinmaya Sachan for the opportunity to conduct this thesis in the Learning and Reasoning Lab (LRE).

Deciding what makes a good Productive Failure task turned out to be a rather subjective question, and at times it was really hard to work on something without a clear right answer. Writing a thesis about Productive Failure also meant that I failed productively quite a few times myself, and I am thankful to everyone who supported me during this journey.

Contents

1 Introduction **1**
- 1.1 Productive Failure and Task Design . . . 1
- 1.2 Generating and Evaluating PF Tasks with LLMs . . . 2
- 1.3 Purpose of this Research . . . 2
- 1.4 Thesis Structure . . . 2

2 Background and Related Work **4**
- 2.1 Foundations of Productive Failure . . . 4
  - 2.1.1 Empirical Evidence for Productive Failure . . . 4
  - 2.1.2 Learning Mechanisms and Design Principles . . . 4
- 2.2 Generating Educational Tasks with LLMs . . . 5
  - 2.2.1 Multiple-Choice Questions and Distractors . . . 5
  - 2.2.2 Applying LLMs to Productive Failure . . . 5
- 2.3 Evaluating Educational Tasks with LLMs . . . 7
  - 2.3.1 Direct Rubric Scoring . . . 7
  - 2.3.2 Decomposed Checklists . . . 7
  - 2.3.3 Knowledge-Profile Student Simulation . . . 7
  - 2.3.4 Prior-Knowledge Leakage in Mathematics . . . 8
- 2.4 Summary and Research Gap . . . 8

3 One-Shot LLM Generation of PF Tasks **9**
- 3.1 Reproducing One-Shot PF Generation . . . 9
  - 3.1.1 Multi-Theme Generation Setup . . . 9
  - 3.1.2 Automated Rubric Judge Setup . . . 10
  - 3.1.3 Rubric Evaluation Results . . . 10
- 3.2 Failure Modes of One-Shot PF Tasks . . . 11
  - 3.2.1 Tasks Solvable by Familiar Methods . . . 12
  - 3.2.2 Step-by-Step Worksheets and Prescribed Representations . . . 12
  - 3.2.3 Off-Target Prompts and Mathematical Contradictions . . . 13
- 3.3 Limitations of Surface Rubric Scoring . . . 14

4 The StructPF Generation Pipeline **16**

4.1 Pipeline Architecture . . . 16
4.2 Stage 1: Prior Knowledge Retrieval . . . 17
4.2.1 Need for Curriculum Grounding . . . 17
4.2.2 The Learning Commons Knowledge Graph . . . 17
4.2.3 Mapping Graph Nodes to Prior Skills . . . 18
4.3 Stage 2: Problem Design and Number Verification . . . 19
4.3.1 Step 2a: Design Brief Selection . . . 19
4.3.2 Step 2b: Task Number Verification . . . 19
4.4 Stage 3: Open Task Writing . . . 20

5 PF EVAL: Measures, Judges, and Validation 22
5.1 From Expert Review to Automated Checks . . . 22
5.2 Five Task-Design Measures . . . 23
5.3 Evaluation Judges and Prior-Knowledge Solver . . . 24
5.3.1 Task Judge . . . 24
5.3.2 Prior-Knowledge Solver . . . 25
5.3.3 Attempt Judge . . . 26
5.4 Validating PF EVAL . . . 27
5.4.1 Solver Recall on Expected Approaches . . . 27
5.4.2 Validating PF EVAL on Reference Tasks . . . 28

6 Experiment Setup and Results 30
6.1 Experiment Setup . . . 30
6.1.1 Study A: PF EVAL Setup . . . 30
6.1.2 Study B: Curriculum Grounding Setup . . . 31
6.2 Study A: PF EVAL Experiment Results . . . 31
6.2.1 One-Shot Tasks under PF EVAL . . . 31
6.2.2 STRUCTPF under PF EVAL . . . 32
6.3 Study B: Curriculum Grounding Experiment Results . . . 33
6.3.1 Step 1: Prior-Knowledge Check . . . 34
6.3.2 Step 2: Method-Failure Check . . . 34
6.4 Negative Results with Simulated Students . . . 36
6.4.1 Classroom Pathway Reproduction . . . 36
6.4.2 Virtual-Classroom Feedback Loop . . . 37

7 Discussion and Conclusion 39
7.1 Revisiting the Research Goals . . . 39
7.2 Implications for LLMs in Education . . . 40
7.3 Limitations and Future Work . . . 42
7.3.1 No Evaluation with Real Students . . . 42
7.3.2 Few Expert-Written PF Tasks . . . 42
7.3.3 Subject and Curriculum Scope . . . 43

7.3.4 Choice of Generator Model 43
7.4 Conclusion 43

A Prompts and Evaluation Rubrics **45**
A.1 Reconstructed One-Shot PF Generation Prompt 45
A.2 Automated Five-Dimension Rubric Judge Prompt 47
A.3 STRUCTPF Prompt Structure 48

List of Figures

1.1 Overview: (a) One-Shot PF Generation and Rubric Scoring (G1) versus (b) StructPF and PF Eval (G2–G4) . . . 3

2.1 One-Shot Prompt (a) and Five-Dimension Rubric (b) from Rahimi et al. . . . 6

3.1 One-Shot Replication: (a) Single Setting in Rahimi et al. versus (b) Four Settings . . . 10
3.2 One-Shot Rubric Scores for GPT-5 mini and GPT-4.1 by (a) Dimension and (b) Setting . . . 11
3.3 One-Shot Task Solvable by Familiar Methods in the Home and Garden Setting . . . 12
3.4 One-Shot Concept Giveaway and Multi-Part Worksheet in the Basketball Setting . . . 13
3.5 One-Shot Mathematical Contradiction in the Home and Garden Setting . . . 14

4.1 Overview of the Three-Stage StructPF Pipeline . . . 17
4.2 Stage 1: Prior Knowledge Retrieval for Grade 6 Polygon Area . . . 18
4.3 Stage 2: Design Brief Selection (a–b) and Number Verification (c) for Grade 6 Polygon Area 19
4.4 Stage 3: Information Barrier and Student-Facing Task for Grade 6 Polygon Area . . . 20

5.1 Mapping Human PF Design Questions to the Five PF Eval Checks . . . 23
5.2 Role-Play Simulated Student (a) versus Target-Blind Prior-Knowledge Solver (b) . . . 23
5.3 Information Separation in PF Eval: (a) Task Judge and (b) Solver with Attempt Judge . . . 25
5.4 Attempt-Judge Labels and Priority Rule for the Limits of Familiar Approaches . . . 26
5.5 Recall of Expected Student Approaches . . . 27
5.6 PF Eval Outcomes on Reference Tasks by Group . . . 28

6.1 Study A Setup: Four Grade 7 Settings with Fixed Prior Knowledge . . . 30
6.2 Study B Setup: Curriculum Topics and Two-Step Check . . . 31
6.3 Surface Rubric versus PF Eval on One-Shot Tasks . . . 31
6.4 Two StructPF Tasks Passing All Five PF Eval Measures (Basketball and Shopping) . . . 33
6.5 Study B Step 1: Prior-Knowledge Check by Generator . . . 34
6.6 Study B Step 2: Method-Failure Check by Generator . . . 35
6.7 Kapur and Bielaczyc Classroom Pathways Enacted by Simulated Students . . . 37
6.8 Virtual-Classroom Feedback Loop versus Self-Review . . . 37

List of Tables

2.1 Human Rubric Scores for GPT-4o and Phi-3 mini (Rahimi et al.) . . . 7
2.2 Prior LLM Studies on Educational Task Generation and Evaluation . . . 8

3.1 Method Giveaways and Worksheets in One-Shot Tasks . . . 13
3.2 Off-Target Prompts and Mathematical Concerns in One-Shot Tasks . . . 14

4.1 Step 2b Calculator Repair Loop for Grade 6 Polygon Area . . . 20

5.1 The Five Task-Design Measures in PF EVAL . . . 24
5.2 Reference Task Groups for Validating PF EVAL . . . 28

6.1 One-Shot Prompting versus STRUCTPF on the Four Grade 7 Settings (Study A) . . . 32

7.1 Calls, Tokens, and Estimated API Cost per Task . . . 43

A.1 Reconstructed One-Shot PF Prompt (Six Parts) . . . 46
A.2 Automated Five-Dimension Judge Prompt (Adapted from Rahimi et al.) . . . 47
A.3 Step 2a Brief Proposal Prompt (PROPOSE_BRIEF) . . . 48
A.4 Step 2a Brief Selection Prompt (SELECT_BRIEF) . . . 48
A.5 Step 2a Quantity Translation Prompt (TRANSLATE_BRIEF) . . . 49
A.6 Step 2b Number Proposal Prompt (PROPOSE_DATA) . . . 49
A.7 Step 2b Number Repair Prompt (REPAIR_DATA) . . . 49
A.8 Stage 3 Task Writing Prompt (WRITE_DATA_TASK) . . . 49

Chapter 1

Introduction

1.1 Productive Failure and Task Design

In traditional classroom instruction, teachers first explain a new concept and its procedures, and students then practice applying them on routine exercises. Productive Failure (PF) reverses this order by dividing learning into two phases [5, 7]. In the first phase, students work on a challenging, uninstructed problem using only the concepts and methods they previously have been taught. In the second phase, the teacher compares the students' attempts with the target concept to show why the new approach is needed. Even though students usually fail to find the canonical solution on their own in the first phase, trying out their existing methods helps them notice where their current knowledge falls short [14]. This prepares them to understand the new concept more deeply and transfer it to new problems across mathematics, science, and engineering [5, 7, 24].

Despite these benefits, Productive Failure is still difficult to use regularly in classrooms because designing a working PF problem requires significant time and effort from teachers [7]. Unlike a standard textbook exercise that simply asks students to apply a taught formula, Kapur and Bielaczyc's PF design framework [7, 8] shows that an effective Phase 1 task must satisfy three requirements at the same time:

1. The problem must be accessible enough for students to start using familiar methods and generate multiple approaches without becoming frustrated and giving up.

2. The numbers in the problem must be deliberately chosen so that those familiar methods fail to solve the problem [7]. For example, when introducing consistency (variance), giving three players the exact same mean and range ensures that students cannot simply compare averages to decide who is more consistent [14].

3. The problem text must stay open-ended and avoid step-by-step hints or formulas so that students have to invent their own approaches, which goes against how teachers and textbooks usually guide students [7].

In practice, finding a conceptual structure, numbers, and wording that satisfy all three requirements takes multiple rounds of designing the problem, testing it on students in class, and adjusting the task [7].

1.2 Generating and Evaluating PF Tasks with LLMs

Large Language Models (LLMs) offer a promising way to reduce this design effort by automatically generating PF problems for a given target concept, with mathematics as a primary testbed in recent work [19–21]. While one-shot prompting could be employed for this purpose, LLMs are trained to solve problems directly and explain concepts step by step rather than to design failure traps. As a result, when asked to write a PF task in one pass, an LLM typically produces a routine story problem that students can already solve with familiar methods, or guides the student step by step toward the target formula, as we observe in Chapter 3. Evaluating these generated PF problems is a second challenge. Normally, assessing whether a PF task is well designed requires teachers or educational experts to review each problem by hand [21], which is hard to conduct at a large enough scale to evaluate machine-generated tasks representatively. At the same time, existing PF rubrics were written for human raters using broad, subjective descriptions rather than concrete checks of the problem's underlying structure and numbers. When these rubrics are used directly for automatic evaluation, they rate the surface of a task, such as an engaging story and open-sounding wording, rather than the three PF requirements described above (Section 3.3).

1.3 Purpose of this Research

Instead of asking an LLM to write a complete PF task in one pass, an alternative approach would be to let it follow the design decisions a teacher makes and to check each decision before the final task is written. In the same way, a generated task can be evaluated by testing the three PF requirements directly rather than by rating a general impression of its text. The research presented in this thesis, illustrated in Figure 1.1, is based on these points and aims to address the following goals:

G1: Reproduce one-shot LLM generation of PF tasks from prior work, and identify where both the generated tasks and their rubric-based evaluation fall short.

G2: Design STRUCTPF, a pipeline that generates PF tasks in smaller, checked steps instead of one pass.

G3: Develop PF EVAL, an evaluation that tests the three PF requirements directly, and validate it on expert-designed and control tasks.

G4: Compare STRUCTPF with one-shot generation across many mathematics topics rather than a single one.

1.4 Thesis Structure

The remainder of this thesis is structured as follows. Chapter 2 reviews the background on Productive Failure and LLMs in education. Chapter 3 addresses G1. Chapter 4 presents STRUCTPF (G2), and Chapter 5 introduces and validates PF EVAL (G3). Chapter 6 describes the experiment setup and compares STRUCTPF with one-shot generation (G4). Finally, Chapter 7 discusses the findings, limitations, and future work.

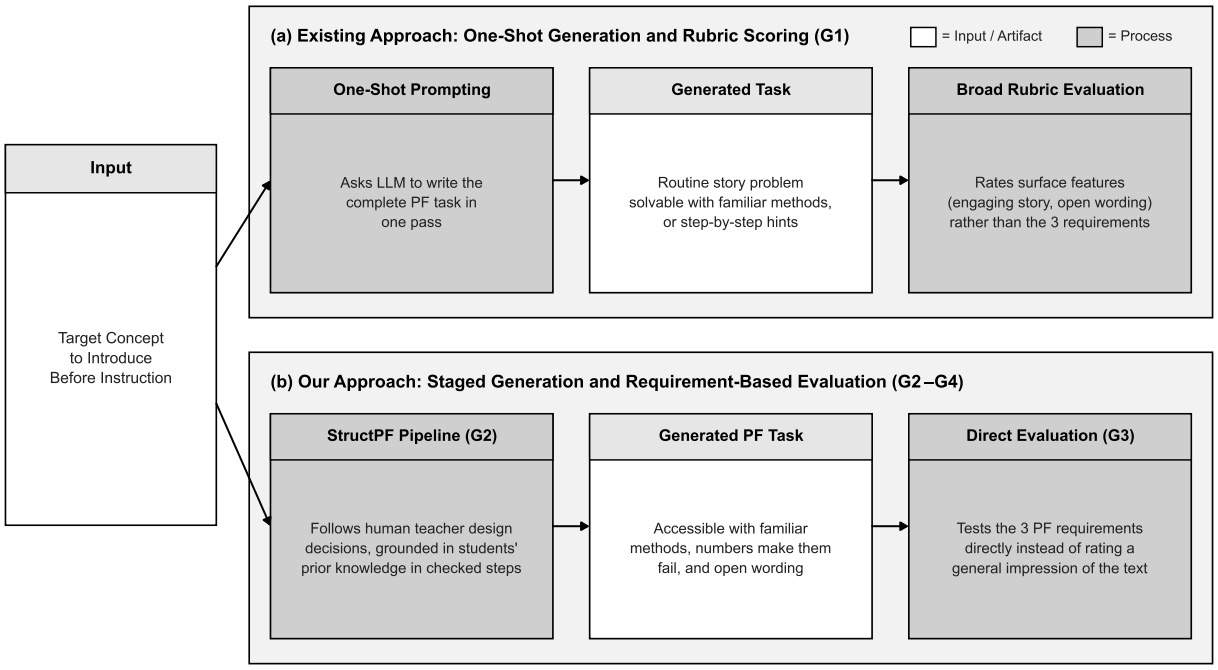


Figure 1.1: Overview: (a) One-Shot PF Generation and Rubric Scoring (G1) versus (b) STRUCTPF and PF EVAL (G2–G4)

Chapter 2

Background and Related Work

2.1 Foundations of Productive Failure

2.1.1 Empirical Evidence for Productive Failure

Kapur first introduced Productive Failure in a study on Newtonian kinematics [5], where Grade 11 students worked in small groups on either ill-structured physics problems, which have missing information and several possible solutions, or well-structured problems. The groups working on ill-structured problems produced poorer solutions at first, but afterward they solved new problems of both types better than the other groups. Later studies moved from comparing two kinds of problems to comparing two whole teaching procedures, namely PF and direct instruction, where the formula is taught first. Kapur and Bielaczyc made this comparison for average speed [7], and Kapur repeated it for variance [6]. These studies showed that the time spent struggling did not come at a cost: PF students calculated with the formula as well as students who had been taught it first, and they were even better at explaining why the formula works. A meta-analysis by Sinha and Kapur [24] confirmed this advantage across many studies and found it to be larger when the problem-solving phase closely follows PF design principles.

2.1.2 Learning Mechanisms and Design Principles

Kapur and Bielaczyc [7] explain why PF works through four interdependent mechanisms, which describe the learning processes that PF aims to trigger in students. They derive the PF design principles from these mechanisms. Besides the requirements for the problem and the comparison of solutions in the second phase, which we described in Section 1.1, these principles ask that students get room to explain and elaborate on their ideas. Each mechanism is supported by one or more of the principles.

(a) *Activating prior knowledge:* Students apply what they already know to the problem and produce several different representations and solution methods. The problem itself supports this by building on that knowledge and allowing many approaches.

(b) *Attending to the critical features of the concept:* Students' attention is drawn to the features that the target concept must capture, such as the spread of data around the mean for variance. All three

principles support this mechanism.

(c) *Explaining and elaborating on these features:* Students explain and elaborate on these features, first in their small groups while working on the problem and later in the whole-class discussion of their solutions.

(d) *Assembling these features into the target concept:* Students organize these features into the target concept, which mainly happens when the teacher compares their solutions with the canonical one.

For these four mechanisms to work, the problem must elicit several distinct attempts that each fail for a clear reason. Classroom studies show that students learn more when they generate more solutions in the first phase [9] and when the teacher can point to the exact flaw in each attempt during the second phase [14]. Because group work and the classroom discussion remain the teacher's responsibility [7], this thesis focuses on generating and evaluating the written problem itself.

2.2 Generating Educational Tasks with LLMs

2.2.1 Multiple-Choice Questions and Distractors

An earlier use case of LLMs in educational task generation is creating multiple-choice questions (MCQs), where each wrong option, or distractor, is designed to match the answer a student reaches through a specific misconception rather than a random guess [1]. Large classroom datasets such as Eedi [27] support this by providing thousands of math questions in which every distractor is paired with a teacher-written explanation of the underlying misconception.

Building on Eedi, Feng et al. [4] tested how well LLMs can generate three distractors per question and measured the share of generated distractors that matched the three human-written Eedi options. When prompted with random example questions, the model matched only 23.6% of the human distractors, and selecting from hand-written error rules or asking GPT-4 to reason step by step about likely student errors (CoT) brought only modest improvements (27–30%). The highest match rate, 38.5%, was achieved by retrieving the three most similar training questions together with their teacher-written distractors and error explanations as few-shot examples (RAG). Even on routine questions, an LLM therefore cannot reliably guess student mistakes from direct prompting alone, and instead needs real examples of student errors.

2.2.2 Applying LLMs to Productive Failure

More recently, researchers have started to bring LLMs to Productive Failure itself. Puech et al. [19] introduced STRATL to steer an LLM tutor during the first phase of PF so that it selects pedagogical intents, such as asking students to test or compare their ideas, instead of revealing the canonical solution. In a high-school field study on expert-designed PF tasks, steering the tutor more than doubled the average number of student-generated representations and solution methods on the consistency task (1.0 to 2.6) and substantially raised students' independent reasoning scores compared with an unsteered LLM tutor. However, their system still requires a human-designed PF problem and its solution space to be given in advance.

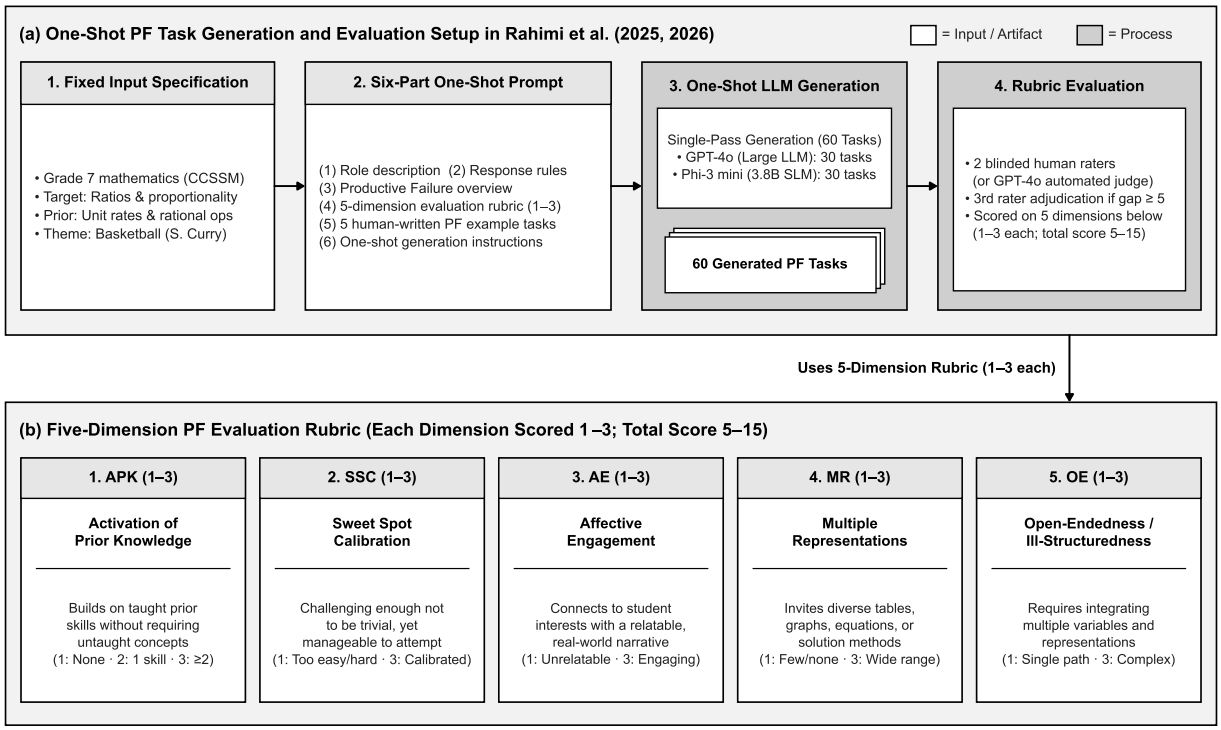


Figure 2.1: One-Shot Prompt (a) and Five-Dimension Rubric (b) from Rahimi et al. [20, 21]

To generate the PF problems themselves, Rahimi et al. [20] developed *ProductiveMath*, an application that prompts GPT-4o to create algebra PF problems from a single six-part prompt containing PF principles, the evaluation rubric, and five human-written PF examples (Figure 2.1a), and rates each generated problem from 1 to 3 across the five rubric dimensions shown in Figure 2.1b for a total score between 5 and 15. Across three studies, they found high agreement between human raters and GPT-4o acting as an automated rubric judge, although seven math teachers who reviewed the problems noted in interviews that the tasks still needed better difficulty calibration, simpler wording, and stronger PF support.

Because *ProductiveMath* relied on a single large model (GPT-4o), Rahimi et al. [21] next tested whether a small language model could match it under the same six-part prompt and rubric. They compared GPT-4o and Phi-3 mini (3.8B) on 60 problems (30 per model) generated for a single fixed setting targeting Grade 7 ratios and proportional relationships, with unit rates and rational-number operations as prior knowledge and a Stephen Curry basketball theme. As summarized in Table 2.1, two blinded human raters found that GPT-4o scored significantly higher than Phi-3 mini overall (13.38 vs. 10.67 out of 15, $d = 1.16$). While both models already scored high on affective engagement and prior-knowledge activation, GPT-4o more reliably followed the prompt's instructions to ask for diverse representations (such as tables, diagrams, and equations) and multi-path problem structures, driving large gains on multiple representations ($d = 1.65$) and open-endedness ($d = 1.58$).

Table 2.1: Human Rubric Scores for GPT-4o and Phi-3 mini (Rahimi et al. [21])

Rubric Dimension	**GPT-4o** ($n = 30$)	**Phi-3 mini** ($n = 30$)	**Cohen's** d
Grade 7 ratios and proportional relationships, basketball theme; mean ± SD across two human raters			
Activation of Prior Knowledge (APK, 1–3)	2.65 ± 0.35	2.42 ± 0.53	0.52
Sweet Spot Calibration (SSC, 1–3)	2.52 ± 0.55	2.08 ± 0.62	0.74
Affective Engagement (AE, 1–3)	2.75 ± 0.47	2.45 ± 0.71	0.50
Multiple Representations (MR, 1–3)	2.70 ± 0.43	1.83 ± 0.61	1.65
Open-Endedness (OE, 1–3)	2.77 ± 0.47	1.88 ± 0.64	1.58
Total Score (5–15)	**13.38 ± 1.88**	**10.67 ± 2.73**	**1.16**

2.3 Evaluating Educational Tasks with LLMs

2.3.1 Direct Rubric Scoring

Once educational tasks are generated automatically, evaluating every output with human teachers becomes a bottleneck. A common alternative is to use an LLM as an automated judge that reads the generated text and scores it on a Likert scale for each rubric dimension, a general evaluation approach popularized by *G-Eval* [13] and used for PF problems in *ProductiveMath* [20]. However, Liu et al. [13], Zheng et al. [28], and Wang et al. [26] show that such rubric-based LLM judges favor LLM-generated text and suffer from strong verbosity, formatting, and position biases, rewarding longer and more polished text even when the underlying content is flawed.

2.3.2 Decomposed Checklists

To make automated text evaluation more reliable, recent work replaces broad Likert scales with decomposed checklists. In *CheckEval*, Lee et al. [11] break each evaluation dimension into specific yes-or-no questions that the judge answers individually before scores are aggregated, which improves correlation with human raters and reduces scoring variance compared with direct Likert scales. Cook et al. [2] report a similar advantage, showing that checklist items help judges catch specific omissions that direct rubric scores overlook. In education, Moore et al. [17] used GPT-4 to evaluate multiple-choice questions against a 19-item writing checklist, matching human raters on over 80% of the criteria. However, their checklist only evaluates surface writing rules, such as clear phrasing and consistent option lengths, rather than the underlying mathematical structure or pedagogical design of the problem.

2.3.3 Knowledge-Profile Student Simulation

Instead of only inspecting the wording of a task, a second line of work evaluates educational items by having LLM-simulated students attempt them. In *Generative Students*, Lu and Wang [15] construct student profiles by specifying which knowledge components a learner has mastered, confuses, or does not know, and prompt simulated students to answer multiple-choice questions on user-interface design, matching college students' item-level performance well ($r = 0.72$). In a similar vein, Scarlatos et al. [22] in *SMART* align simulated student responses with Item Response Theory to estimate the difficulty of reading and programming

questions on held-out test items.

2.3.4 Prior-Knowledge Leakage in Mathematics

When this persona-based approach is applied to mathematics, however, capable language models struggle to stay within a student's actual prior knowledge. Liu et al. [12] tested GPT-4 on Eedi math questions and found that while the model solved 94.8% of the problems correctly as an expert tutor, it could reproduce a specified student misconception on only 61.7% of the items, often defaulting back to the correct calculation. Srivatsa et al. [25] similarly found on national mathematics assessments (NAEP) that no model or persona prompt reliably matched real students' grade-level ability boundaries. Furthermore, Do, Sonkar, and Sachan [3] showed that LLM student simulators abandon their initial errors under almost any follow-up prompt, even when the feedback is generic or unrelated to their mistake. Role-play student personas therefore cannot be trusted on their own to respect strict prior-knowledge limits on open math problems.

2.4 Summary and Research Gap

Table 2.2: Prior LLM Studies on Educational Task Generation and Evaluation

Study	Focus	Open PF Tasks	Prior Knowledge	Method-Failure Check
Feng et al. [4]	MCQ distractors	–	✓ (error RAG)	–
Lu and Wang [15]; Liu et al. [12]	Student simulation	–	✓ (persona profile)	–
Puech et al. [19] (STRATL)	PF Phase 1 tutoring	–	✓ (expert-supplied)	–
Rahimi et al. [20, 21]	PF task generation	✓ (one-shot)	–	– (surface rubric)
This thesis (STRUCTPF)	**PF task generation**	**✓ (3-stage)**	**✓ (curriculum graph)**	**✓ (calculator + solver)**

Table 2.2 summarizes how prior work relates to this thesis. On the generation side, MCQ distractor studies such as Feng et al. [4] show that LLMs cannot reliably guess student mistakes from direct prompting alone and perform much better when given external information about student errors. However, MCQ generation only targets single-step mistakes on routine questions and does not capture the complexity of PF, where an open problem must let students try and compare several prior methods. Existing work on PF task generation targets open problems directly, but has only studied one-shot prompting on a narrow set of Grade 7 algebra topics, relying on the model's general ability to invent the scenario, numbers, and wording in a single pass [20, 21]. On the evaluation side, prior PF studies score generated tasks on a surface level with a broad 1–3 rubric without testing whether familiar methods actually fail on the problem's numbers, while unconstrained role-play student simulators often solve problems using knowledge beyond the student's grade [12, 25]. Given these limitations of one-shot prompting, surface rubrics, and unconstrained student simulation, this thesis investigates whether decomposing task design into curriculum-grounded, calculator-checked stages and evaluating the PF requirements directly can produce working PF tasks across a much broader range of mathematics topics and grade levels.

Chapter 3

One-Shot LLM Generation of PF Tasks

3.1 Reproducing One-Shot PF Generation

The high rubric scores reported by Rahimi et al. [21] (13.38 out of 15 for GPT-4o in Table 2.1) suggest at first glance that a single prompt is already sufficient to generate effective PF problems. However, as established in Section 1.1, a working PF problem requires more than an engaging story and open-sounding phrasing, as its numbers must be chosen so that familiar methods fail and its wording must leave the core method open for students to invent. To address our first research goal (G1), we therefore examine whether one-shot generated tasks actually satisfy these requirements, or whether the five-dimension rubric rewards surface features while overlooking structural flaws.

3.1.1 Multi-Theme Generation Setup

Answering this question requires inspecting the generated tasks and their rubric ratings directly. Rahimi et al. [21] published the six-part structure of their prompt, the verbatim five-dimension rubric, and the Grade 7 basketball setting inputs, but did not release the exact prompt wording, the five in-prompt examples, or the generated tasks. We therefore reconstruct the six-part prompt from their published specification and rubric using five human-written PF tasks from the STRATL collection [19] (Table A.1), and extend the experiment from their single Grade 7 basketball ratio setting to three additional Grade 7 settings covering percents, random-sample distributions, and composite area (Figure 3.1b).

While Rahimi et al. [21] compared GPT-4o against an early 3.8B small language model (Phi-3 mini), compact models have since gained built-in chain-of-thought reasoning. We therefore compare a compact reasoning model (GPT-5 mini) against a standard flagship chat model (GPT-4.1) to test whether a smaller model with internal reasoning can match or exceed a larger non-reasoning model under the same one-shot prompt. In each of the four Grade 7 settings, we generate 30 tasks with GPT-5 mini and 30 tasks with GPT-4.1 under default sampling without retries, yielding 240 generation runs in total.

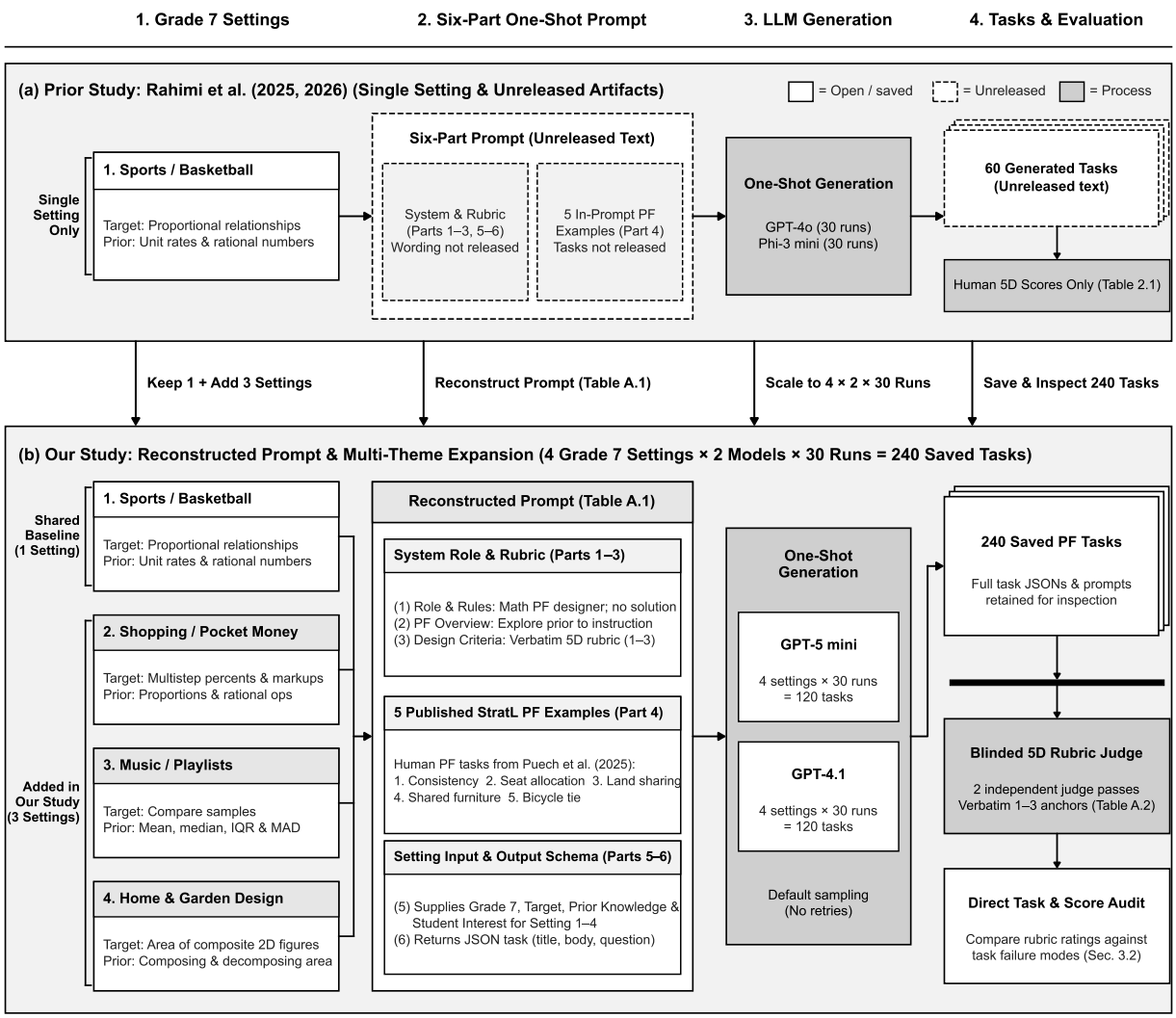


Figure 3.1: One-Shot Replication: (a) Single Setting in Rahimi et al. [21] versus (b) Four Settings

3.1.2 Automated Rubric Judge Setup

To evaluate all 240 generated tasks consistently, we use an LLM judge (GPT 5.6 Luna) built from the five dimensions and 1–3 scoring anchors of Rahimi et al. [21] (full prompt in Appendix A.2, Table A.2), following their use of an automated LLM rubric judge in *ProductiveMath* [20]. We keep their rubric wording unchanged so that our results reflect how the original rubric behaves without extra rules or score tuning. Each task is scored in two independent passes without revealing which model generated it, and the two 5–15 totals are averaged to obtain the final score.

3.1.3 Rubric Evaluation Results

Out of the 240 generation runs, 238 returned valid tasks (118 from GPT-5 mini and 120 from GPT-4.1), with 117 and 120 tasks fully scored after one judge call timed out. Across all four settings combined, GPT-5 mini

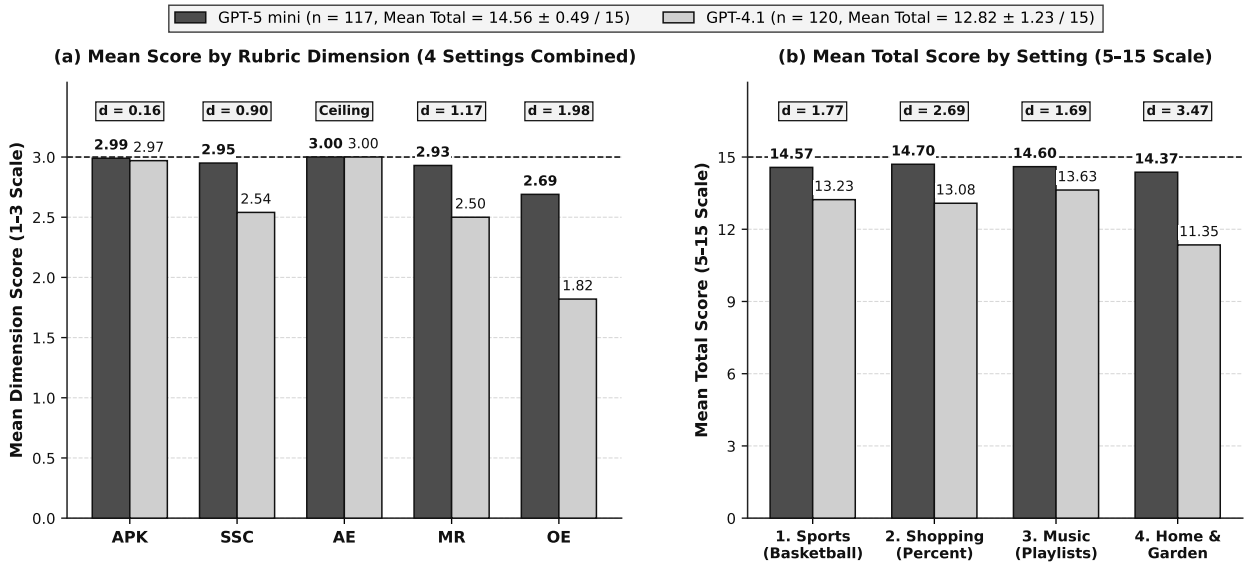


Figure 3.2: One-Shot Rubric Scores for GPT-5 mini and GPT-4.1 by (a) Dimension and (b) Setting

achieves a mean total score of 14.56 ± 0.49 out of 15, outperforming GPT-4.1 at 12.82 ± 1.23 with a large effect size ($d = 1.83$). Both models therefore score in the same high range as the 13.38 total reported for GPT-4o in Table 2.1, with GPT-5 mini approaching the 15-point ceiling.

As shown in Figure 3.2a, both models hit the 3.0 ceiling on affective engagement and activation of prior knowledge, showing that any task written around the assigned theme and prerequisites satisfies these two criteria. Consistent with the human ratings in Rahimi et al. [21], the gap between the two models comes almost entirely from open-endedness (2.69 vs. 1.82), multiple representations (2.93 vs. 2.50), and sweet spot calibration (2.95 vs. 2.54). Across the four Grade 7 settings in Figure 3.2b, GPT-5 mini stays near the 15-point ceiling on every setting, whereas GPT-4.1 scores above 13.0 on three settings but drops to 11.35 on Home and Garden because it ignores the PF design principles in the prompt and gives away the solution setup directly in the task text. At first glance, these high rubric scores suggest that one-shot prompting with a current reasoning model solves PF task generation across settings, which motivates inspecting the generated tasks directly in the next section.

3.2 Failure Modes of One-Shot PF Tasks

To see whether these high-scoring problems actually work as Productive Failure tasks, we inspect all 238 generated tasks against the PF design requirements from Section 1.1. Despite the high rubric scores in Section 3.1.3, three failure modes recur across all four settings. The following three subsections introduce each failure mode with a concrete task example, measure how often it appears across the 238 tasks, and explain why it motivates the dedicated checks in Chapter 5.

3.2.1 Tasks Solvable by Familiar Methods

The first failure mode is that many one-shot tasks lack a setup where familiar methods fall short, so a routine procedure from an earlier grade solves the problem right away. In the Home and Garden setting, for instance, students have already learned in Grade 6 to find the area of rectangles and right triangles. As illustrated in Figure 3.3, the generated task presents the garden directly as three separate pieces with all bases and side lengths given, so students simply add the three familiar formulas without facing an unpartitioned figure or a missing dimension.

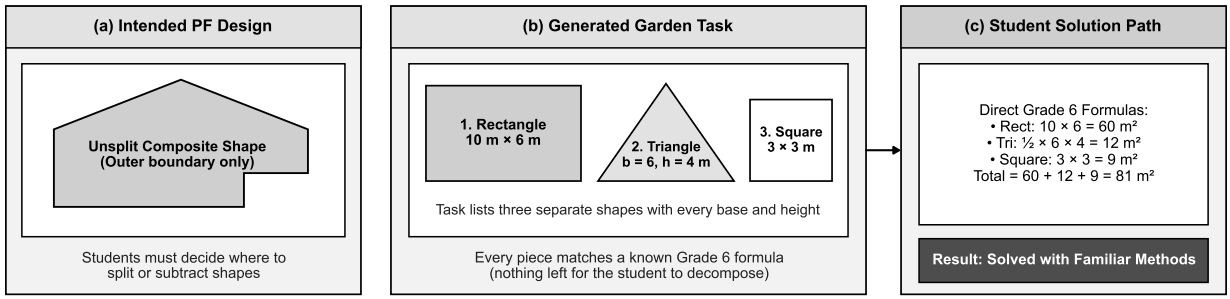


Figure 3.3: One-Shot Task Solvable by Familiar Methods in the Home and Garden Setting

Across all 60 Home and Garden tasks, every single task from both models already presents the garden as named rectangles and triangles rather than an unpartitioned figure, yet the rubric still rates sweet spot calibration high because it only asks whether a task looks challenging and manageable on the page rather than whether familiar methods fail. Moreover, a text search can only spot this issue when a geometry prompt names the simple shapes outright. In numerical topics such as Basketball or Shopping, looking at the words alone cannot tell us whether a familiar calculation already solves the problem. Testing that requires working through the given numbers using only the students' prior knowledge, which motivates the prior-knowledge solver we introduce in Chapter 5.

3.2.2 Step-by-Step Worksheets and Prescribed Representations

The second failure mode is that the prompt guides students through the solution instead of leaving the approach open for them to invent, primarily in GPT-5 mini. As illustrated in Figure 3.4, the generated basketball task announces the untaught target concept in the opening paragraph and splits the question into a three-part worksheet that tells students what to check in Part 1 and lists five specific representations for them to draw in Part 2.

As shown in Table 3.1, GPT-5 mini regularly coaches students through both hints and worksheet structure. Over 80% of its questions name the target concept or suggest specific representations, and 84.7% split the question into three or more numbered parts, more than doubling the median task length to 422.5 words. Because the generation prompt includes the evaluation rubric (Table A.1), GPT-5 mini satisfies the top anchors for multiple representations and open-endedness literally by naming representations and stacking multi-part questions. Instead of penalizing this over-guidance, the rubric rewards it with higher scores, which motivates a dedicated check in Chapter 5 for whether a task leaves the solution method open to the

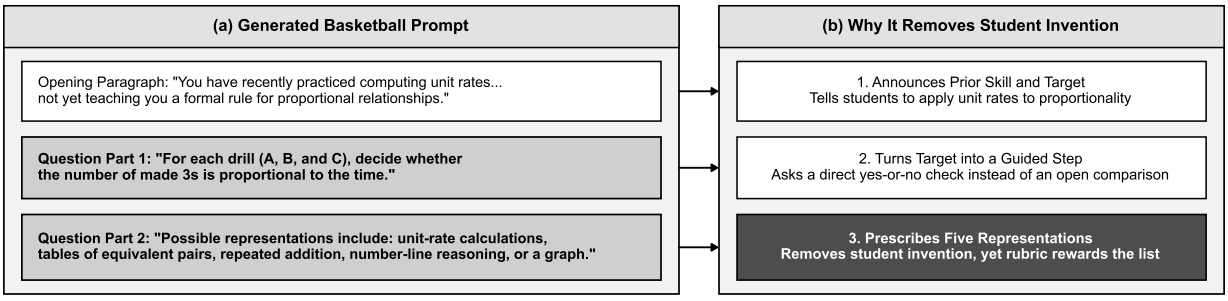


Figure 3.4: One-Shot Concept Giveaway and Multi-Part Worksheet in the Basketball Setting

student.

Table 3.1: Method Giveaways and Worksheets in One-Shot Tasks

Observed Issue Pattern	**GPT-5 mini** ($n = 118$)	**GPT-4.1** ($n = 120$)
Basketball questions naming *"proportional"*	24/30 (80.0%)	0/30 (0.0%)
Music questions naming center or spread measures	25/29 (86.2%)	8/30 (26.7%)
Questions prescribing specific representations or tools	98/118 (83.1%)	3/120 (2.5%)
Questions split into a multi-part worksheet (≥ 3 parts)	100/118 (84.7%)	0/120 (0.0%)
Median task length (words)	422.5 words	200.5 words

3.2.3 Off-Target Prompts and Mathematical Contradictions

The third failure mode is that generated tasks often miss part of the target concept or contain contradictions in their numbers and geometry. In the garden task shown in Figure 3.5, students are asked to find the area and outer border of a garden made of a central 12 ft $\times$ 10 ft rectangle with a triangle and a trapezoid attached to its sides. The opening story says that the triangle and the trapezoid sit on opposite long sides of the rectangle, but the measurement list below it attaches the triangle to a 10-ft short side and the trapezoid to a 12-ft long side (Figure 3.5a). Because a 10-ft side and a 12-ft side meet at a corner rather than facing each other, the two shapes cannot be on opposite sides, so students cannot draw the garden or find its border consistently (Figure 3.5b).

Table 3.2 reports the counts for both task soundness issues. Missing the target concept is clearest in the Music setting, where the Grade 7 goal is to draw conclusions about two large catalogs from small random samples, whereas 90.0% of GPT-4.1 tasks never mention sampling and only ask students to compare two short lists directly, turning the problem back into a Grade 6 exercise. Across all four settings, the rubric judge itself logs at least one entry in its separate mathematical-concerns list (Table A.2) on over 90% of tasks from both models without lowering the rubric score. Because a single concern list mixes genuine contradictions with minor wording notes and never lowers the rubric score, Chapter 5 introduces dedicated task soundness checks for target alignment and mathematical validity.

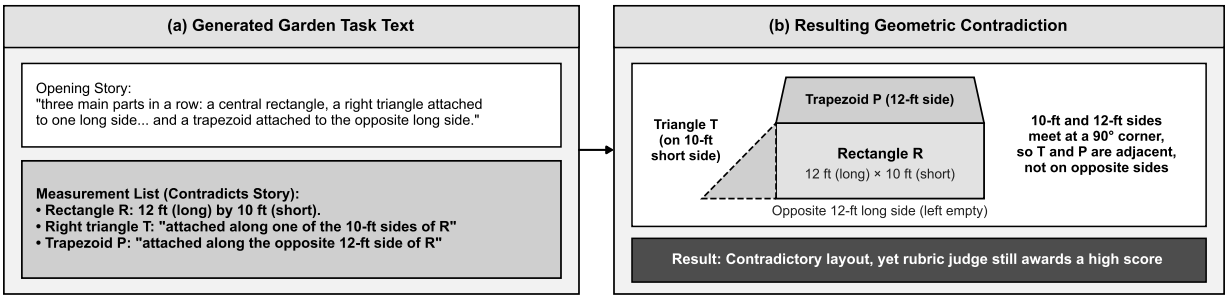


Figure 3.5: One-Shot Mathematical Contradiction in the Home and Garden Setting

Table 3.2: Off-Target Prompts and Mathematical Concerns in One-Shot Tasks

Observed Issue Pattern	**GPT-5 mini** ($n = 118$)	**GPT-4.1** ($n = 120$)
Music tasks omitting random-sample framing	2/29 (6.9%)	27/30 (90.0%)
Tasks with ≥ 1 judge-logged mathematical concern	111/118 (94.1%)	108/120 (90.0%)

3.3 Limitations of Surface Rubric Scoring

The gap between the high rubric scores in Section 3.1 and the flaws in Section 3.2 comes from how the five 1–3 scoring anchors are written (Table A.2). Because the anchors describe general impressions of the text rather than concrete mathematical and structural checks, a generated task can satisfy all five dimensions on the surface while failing the three PF requirements from Section 1.1 for two main reasons.

1. **Wording Anchors Reward Hints and Worksheets**

   Affective engagement and activation of prior knowledge only require a realistic story and two prerequisite skills, so both reach the ceiling even when a task omits part of the target standard or gives away the formula. For multiple representations and open-endedness, the rubric actively rewards the wrong behavior, in line with known length and formatting biases in LLM judges [26, 28]. Because the top anchors ask for a wide range of representations and multiple variables, a model can raise its score simply by naming which representations students should use and splitting the prompt into a multi-part worksheet. The very instructions that remove student invention are therefore rewarded with higher rubric scores.

2. **Text-Only Scoring Never Tests the Math or Familiar Methods**

   A second limitation is that the rubric judge only reads the problem text and never works through the numbers or geometry. The sweet spot calibration anchor only asks whether a task looks challenging but manageable on the page, without testing whether familiar methods actually fail. As a result, tasks that already name the component rectangles and triangles directly or contain impossible geometry (Figure 3.5) still receive high rubric scores even when the judge records an issue in its separate mathematical-concerns list.

Together, these findings answer our first research goal (G1). Even with modern chat and reasoning models, one-shot generation and direct rubric scoring still treat Productive Failure as a writing style rather than a

mathematical design problem. When a single prompt asks a model to write a story, pick the numbers, and follow the rubric at once, even a stronger reasoning model simply optimizes for surface features that are easy to write and easy for a text-only judge to reward. Addressing both issues requires breaking task design and evaluation into separate, checkable steps. In Chapter 4, we introduce STRUCTPF (G2) to address the generation side by separating prior-knowledge grounding, choosing and checking numbers where familiar methods fail, and writing the final task without hints, before introducing PF EVAL (G3), our requirement-based evaluation, in Chapter 5.

Chapter 4

The StructPF Generation Pipeline

4.1 Pipeline Architecture

To create a problem that triggers Productive Failure, a human designer does not write a story and choose its numbers in a single step. Instead, as documented by Kapur and Bielaczyc in their design of the Grade 7 average-speed task [7], designing a working PF problem follows a structured sequence of four decisions.

1. **Identifying prior knowledge:** The designers first inspect earlier curriculum grades to list the concepts and methods students have already been taught, such as speed and ratios from Grades 1 to 6, so that students have familiar tools to start the problem (Requirement 1 in Section 1.1).

2. **Designing the problem structure:** Next, they choose a problem situation around the critical features of the target concept and anticipate the specific methods students will try using their prior knowledge.

3. **Testing and adjusting the numbers:** They then test candidate numbers in pilot sessions and adjust the values so that the target concept holds while familiar shortcut methods fail to give the right answer (Requirement 2).

4. **Writing an open problem statement:** Finally, they write the student-facing story and question so that the goal and given quantities are clear, while the solution method is left open for students to invent and explain (Requirement 3).

As we saw in Chapter 3, asking an LLM to make all four decisions in a single prompt fails because the model already knows the target formula. It either chooses convenient numbers that familiar shortcuts solve immediately or coaches the student step by step toward the canonical procedure. We have therefore chosen to structure StructPF around the same sequence of steps that a human designer follows, because each decision targets a specific PF mechanism from Section 2.1.2 and requires a different kind of check. In order to prevent the model from leaking the target formula or optimizing for the grading criteria, we also keep all planning records and canonical solutions in internal design notes and withhold both the key idea and the evaluation rubric from the final task writer.

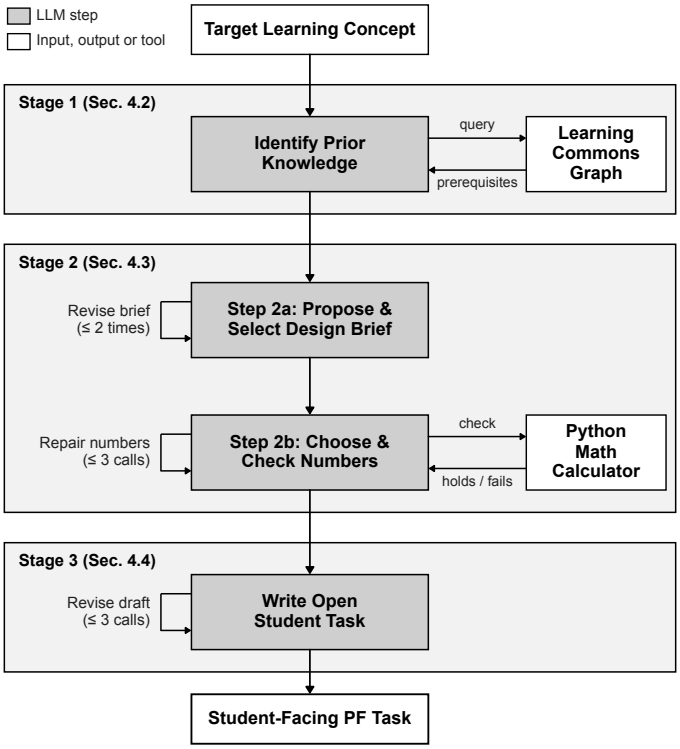


Figure 4.1: Overview of the Three-Stage STRUCTPF Pipeline

Figure 4.1 illustrates the resulting three-stage STRUCTPF pipeline, which mirrors the human design workflow and uses a fixed revision limit at each stage rather than retrying indefinitely. The following three sections introduce each stage in turn and trace the Grade 6 polygon-area standard (`6.G.A.1`) through the entire pipeline.

4.2 Stage 1: Prior Knowledge Retrieval

4.2.1 Need for Curriculum Grounding

Stage 1 must identify the valid methods from earlier grades that students can apply before the target concept is taught. When asked directly for prerequisites, however, an LLM already knows the target concept and does not reliably separate earlier-grade skills from it, as we show in Section 6.3. On a Grade 8 linear-function topic, for example, prompting without curriculum data treated the two-point slope formula $(y_2 - y_1)/(x_2 - x_1)$ as already known rather than keeping the earlier proportional ratio check y/x, giving away the core calculation before the task was even written. Stage 1 therefore requires an external curriculum source that explicitly records which skills precede each target standard.

4.2.2 The Learning Commons Knowledge Graph

For this curriculum source, we use the open Learning Commons Knowledge Graph (`v1.14.0`) [10], which connects US K–12 Common Core mathematics standards into a directed graph. As shown on the left of

Figure 4.2, the graph provides two types of links around each standard. First, `buildsTowards` links connect earlier-grade standards directly to the later-grade standard they prepare students to learn. Second, `supports` links break each standard down into smaller skills that state what a student can actually calculate. Together, these two links tell Stage 1 both which earlier standards are allowed and which concrete calculations students can try at the start of a problem.

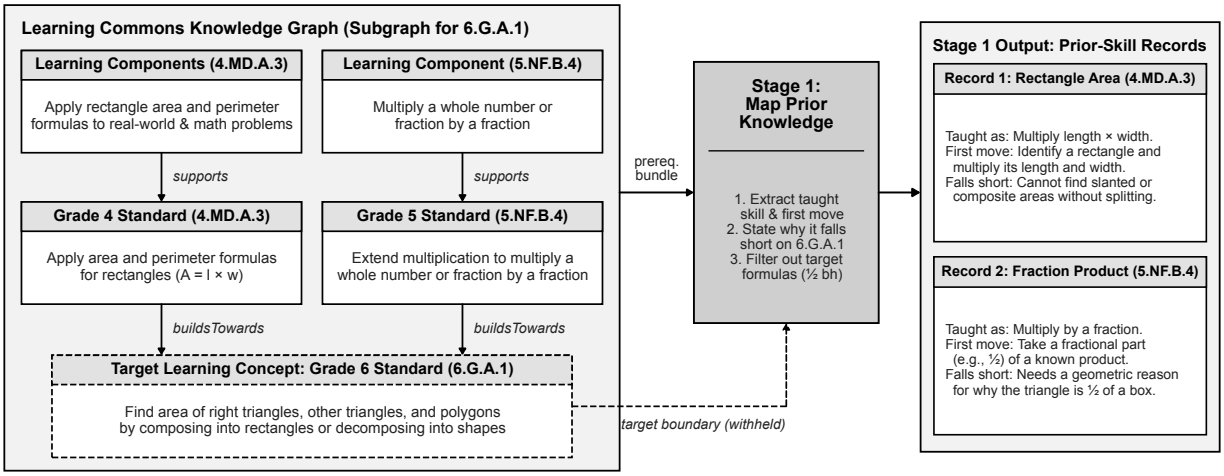


Figure 4.2: Stage 1: Prior Knowledge Retrieval for Grade 6 Polygon Area

4.2.3 Mapping Graph Nodes to Prior Skills

An illustration of the Stage 1 process is shown in Figure 4.2. Given a target Common Core standard, the pipeline follows incoming `buildsTowards` edges one hop backward to retrieve its direct prerequisite standards and their attached `supports` learning components, averaging 2.6 parent standards per target across our 30 benchmark topics. We stop at one hop because deeper traversals reach basic elementary arithmetic that is too general to design a focused PF problem. However, raw curriculum nodes only state a general skill, such as Grade 4 rectangle area, without explaining how a student would try to use it on the new topic or why it fails. Stage 1 therefore passes the retrieved prerequisite nodes and the target topic boundary to an LLM, which turns each node into a structured prior-skill record as shown on the right of Figure 4.2. Each record states how the skill was taught in earlier grades, the initial calculation a student can try, and why that calculation falls short without the target concept. For example, on Grade 6 polygon area, the model turns Grade 4 rectangle area into multiplying length by width as a first move that cannot find slanted or composite areas without splitting, while any candidate that already uses the Grade 6 triangle formula $\frac{1}{2}bh$ is filtered out.

When a topic is given only as free text or comes from a curriculum outside US Common Core, such as Singapore Grade 7 average speed or German Grade 10 variance, Stage 1 queries the Learning Commons search API with the topic phrase to select the closest matching standards, such as Common Core unit-rate standards. If the graph has no close progression link for the level, as in undergraduate fair division, the pipeline instead retrieves grade-filtered excerpts from local curriculum documents or accepts teacher-supplied prerequisite skills.

4.3 Stage 2: Problem Design and Number Verification

4.3.1 Step 2a: Design Brief Selection

Once Stage 1 has identified what students already know, Stage 2 must design a problem setup where familiar shortcuts fall short and verify its concrete numbers before the problem text is written. In Chapter 3, asking an LLM to write the prompt directly caused all 30/30 Home and Garden tasks to present already-split shapes that routine formulas solved immediately (Figure 3.3). We therefore separate conceptual problem design (Step 2a) from number verification (Step 2b) and task writing (Stage 3), as illustrated for Grade 6 polygon area (`6.G.A.1`) in Figure 4.3.

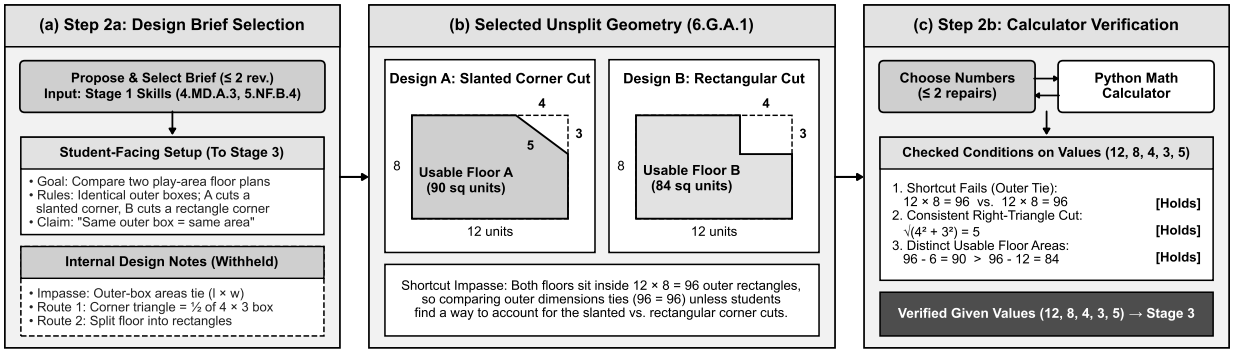


Figure 4.3: Stage 2: Design Brief Selection (a–b) and Number Verification (c) for Grade 6 Polygon Area

Given the Stage 1 prior-skill records, Step 2a generates three independent candidate design briefs (prompts in Tables A.3–A.5) so the pipeline can compare alternative problem structures before choosing numbers. As shown in Figure 4.3a–b, each candidate brief separates the student-facing setup and open question from the withheld internal design notes that record the target key idea, the naive shortcut, and the numerical requirements. A selector model then chooses the candidate that is accessible with Stage 1 skills, causes the simplest Stage 1 shortcut to fall short, and avoids method giveaways in the student-facing fields, returning at most two revision calls if a concern remains before passing the brief to Step 2b.

4.3.2 Step 2b: Task Number Verification

To turn the selected design brief into a working problem, we need to choose concrete numbers for the quantities in the task. The goal is to ensure that the familiar shortcut fails while the geometry and the intended solution remain valid. We have chosen to use a Python calculator for this step because an LLM often picks numbers where a shortcut already works or introduces impossible dimensions when writing a problem directly (Figure 3.5). For this sub-process we use `SymPy` [16] to evaluate arithmetic, square roots, list totals, and equalities or inequalities. In order to keep the final answer out of the student task, we also label each number as either a student-visible given value or a withheld internal value.

An illustration of this verification process is shown in Figure 4.3c. Every selected design brief undergoes this process (prompts summarized in Tables A.6–A.7), which has two parts. In the first part, the model proposes concrete numbers for the quantities in the brief and separates the brief's requirements into com-

putable conditions and qualitative rules. Only the computable conditions are translated into mathematical expressions for the calculator, whereas qualitative rules, such as keeping a figure unsplit, are checked directly against the brief. The second part of the process entails verifying the computable conditions in the Python calculator. If any condition is not satisfied, the calculator returns the computed values and the failed check to the model so that the numbers can be adjusted, repeating this step for at most two repair rounds. Finally, once all conditions hold, the verified student-visible values are passed to Stage 3, as illustrated for the play-area floor task in Table 4.1.

Table 4.1: Step 2b Calculator Repair Loop for Grade 6 Polygon Area

Loop Step	**Proposed Dimensions**	**Calculator Check Result**	**Loop Action**
Initial proposal	Outer boxes 12×8 vs. 10×8; Design A cut $4, 3, 6$; Design B cut 4×3	**Rejected:** Condition 1 fails ($96 \neq 80$, outer shortcut works) and Condition 2 fails ($\sqrt{4^2 + 3^2} = 5 \neq 6$)	Return both failed predicates and computed values for Repair 1
Repair 1	Outer boxes both 12×8; Design A cut $4, 3, 6$; Design B cut 4×3	**Rejected:** Condition 1 holds ($96 = 96$), but Condition 2 still fails ($\sqrt{4^2 + 3^2} = 5 \neq 6$, impossible triangle)	Return failed hypotenuse check for Repair 2
Repair 2 (Final)	Outer boxes both 12×8; Design A cut $4, 3, 5$; Design B cut 4×3	**Accepted:** Conditions 1–3 hold ($96 = 96$, $\sqrt{4^2 + 3^2} = 5$, and floor areas $90 > 84$)	Freeze and pass student-visible values $(12, 8, 4, 3, 5)$ to Stage 3

4.4 Stage 3: Open Task Writing

To complete the task design, we need to turn the verified problem setup and numbers from Stage 2 into a clear student-facing problem while leaving the solution method open for students to invent. The Stage 3 writer prompt is summarized in Table A.8. As we saw in Chapter 3, when an LLM sees the target concept and the five-dimension evaluation rubric while writing the task, it frequently coaches the student by naming specific representations or breaking the problem into a step-by-step worksheet (Figure 3.4). We have therefore chosen to place an information barrier between Stage 2 and Stage 3, because a language model that sees the canonical solution or the grading rubric when writing the task turns internal planning notes into student hints.

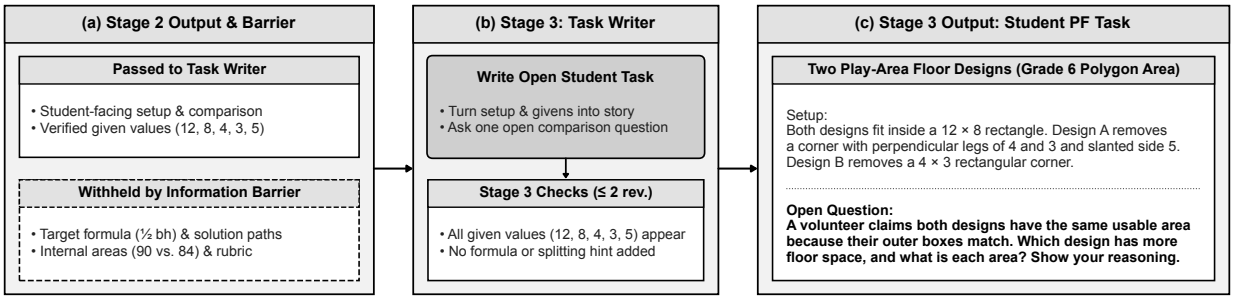


Figure 4.4: Stage 3: Information Barrier and Student-Facing Task for Grade 6 Polygon Area

An illustration of Stage 3 and the resulting task for our Grade 6 polygon-area example is shown in Figure 4.4.

Passing only the student-facing setup and the verified student-visible numbers across the barrier produces a concise task (*"Two Play-Area Floor Designs"*) that defines the comparison clearly while keeping the polygons unsplit and leaving the solution strategy to the student.

By separating prior knowledge retrieval (Stage 1), problem design and number verification (Stage 2), and open task writing (Stage 3), STRUCTPF mirrors the four decisions of a human PF designer while keeping internal planning notes and evaluation rubrics out of the student-facing prompt. To test whether the generated tasks actually resist familiar shortcuts and elicit multiple student approaches, Chapter 5 introduces PF EVAL.

Chapter 5

PF EVAL: Measures, Judges, and Validation

5.1 From Expert Review to Automated Checks

As shown in Chapter 3, a surface rubric judge cannot tell whether a generated problem actually works as a PF task. An expert teacher can spot missing impasses, method giveaways, and mathematical errors by hand, but manual review does not scale across hundreds of generated tasks. We therefore introduce PF EVAL, an automated evaluation framework that tests whether a generated task satisfies the underlying requirements of Productive Failure rather than how convincing its text reads.

Just as STRUCTPF mirrors the four decisions a human designer makes to construct a PF task (Section 4.1), PF EVAL turns the PF requirements from Section 1.1 into the separate questions an expert asks when reviewing a finished task. Each question receives its own outcome, following decomposed checklist evaluations such as CheckEval [11] and TICK [2]. Figure 5.1 summarizes these four review questions and the automated checks that answer them.

Three of the four questions in Figure 5.1 can be checked directly from the task text and its numbers, whereas Requirement 2 cannot, because a judge that only reads the task never tries the familiar methods on the given values. Simulated students address this by letting an LLM attempt each item while playing a student with a given knowledge profile [15, 22], but capable models in mathematics frequently leak knowledge beyond the assigned profile or abandon their errors (Section 2.3.4). More fundamentally, Requirement 2 asks about the task rather than about the students. As illustrated in Figure 5.2a, even when a simulated student stays within its profile, a solved or failed attempt remains ambiguous between student invention, leaked knowledge, and deliberate role-play mistakes. Our prior-knowledge solver (Figure 5.2b) instead tests the task directly by doing its best with only the listed prior procedures and no deliberate errors, while a separate attempt judge that knows the target concept sets aside any attempt that uses an unlisted concept. Section 5.2 defines the five task-design measures behind Figure 5.1, Section 5.3 describes the prior-knowledge solver and the two judges, and Section 5.4 validates PF EVAL as a whole.

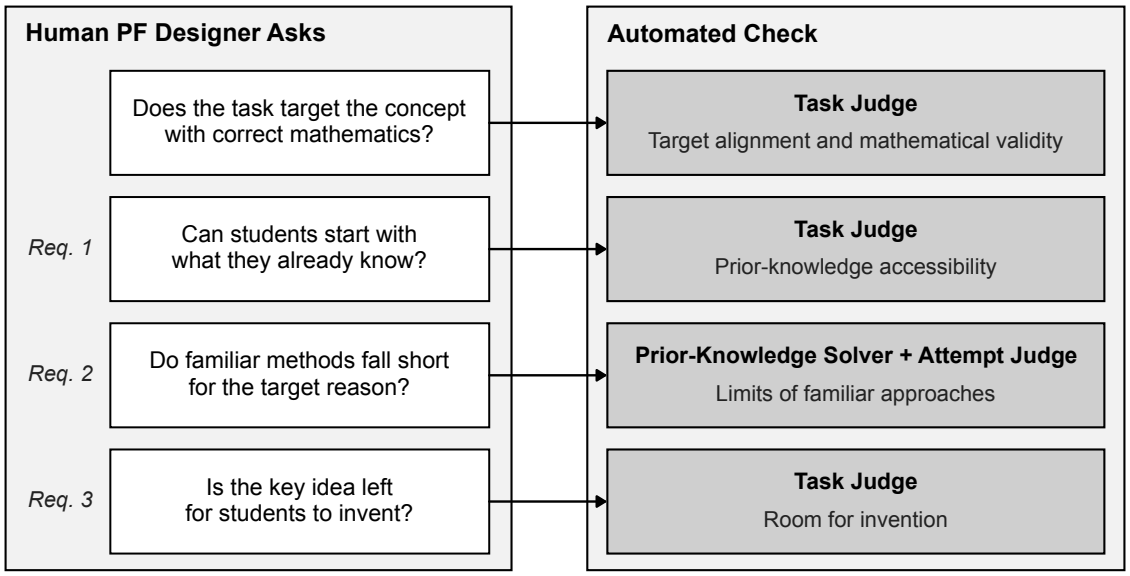


Figure 5.1: Mapping Human PF Design Questions to the Five PF EVAL Checks

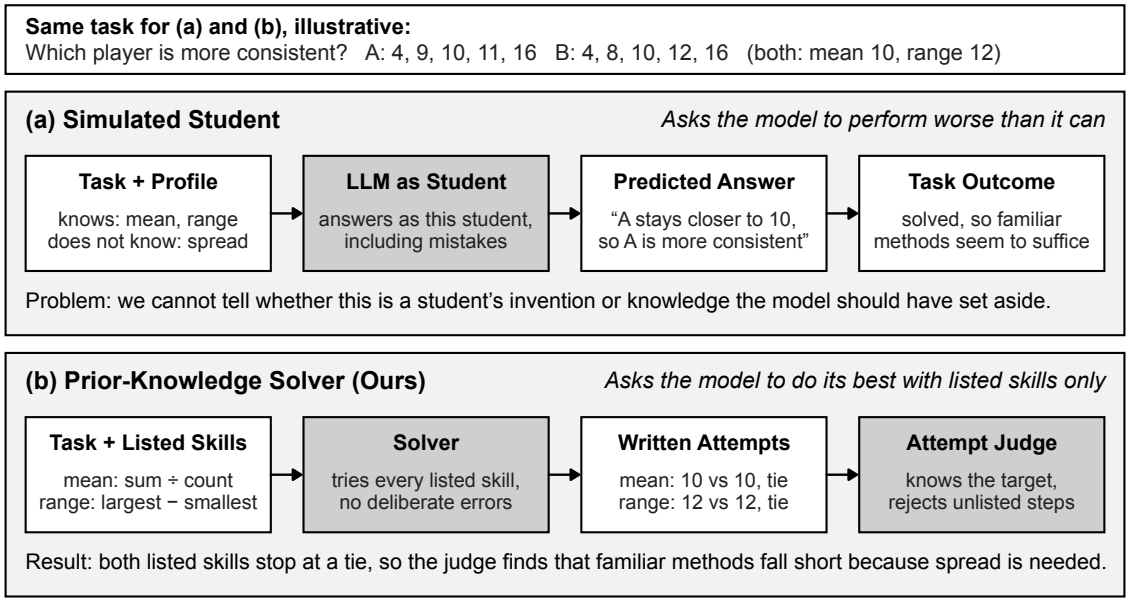


Figure 5.2: Role-Play Simulated Student (a) versus Target-Blind Prior-Knowledge Solver (b)

5.2 Five Task-Design Measures

To evaluate a generated task against the questions in Figure 5.1, we need measures that show which requirement holds and which fails. We have chosen to evaluate each question as a separate measure with its own outcome (*Yes*, *No*, or *Unclear*) rather than summing points into a single score, because a high total can hide a failure on one requirement. Table 5.1 defines the five measures, grouped into two task soundness checks and three PF requirement checks. The two task soundness measures verify that the problem addresses the

target concept with consistent mathematics, since a task that teaches the wrong concept or contradicts itself cannot be productive, while the remaining three test PF Requirements 1 to 3 directly.

Table 5.1: The Five Task-Design Measures in PF EVAL

Measure	**Definition**
Target alignment	Solving the task's central problem requires the target concept, even when the task never names it.
Mathematical validity	The givens are consistent with each other, and the task has at least one solution.
Prior-knowledge accessibility (Req. 1)	A student can make a meaningful start on the task using the listed prior skills and basic arithmetic.
Limits of familiar approaches (Req. 2)	A familiar method, carried out correctly, falls short for the reason that the target concept resolves.
Room for invention (Req. 3)	The task leaves its central construction, such as a measure, model or fair rule, for students to build.

We evaluate the five measures in a fixed order so that a failed task soundness check or an incomplete problem statement is not confused with a failed PF requirement. In particular, we check mathematical validity only when a task is aligned with the target concept, and we test the limits of familiar approaches only when the task is accessible with prior knowledge, because a familiar method cannot meaningfully fall short on a problem that students cannot even start. When an earlier check fails, the dependent measure is recorded as *not assessed* rather than *No*. Similarly, when essential context such as a referenced diagram is missing, we return *Unclear* so that uncertainty in the judge is not counted as a confirmed flaw in the task.

5.3 Evaluation Judges and Prior-Knowledge Solver

To score the five measures, PF EVAL divides the evaluation across the three LLM roles shown in Figure 5.3, where the task judge evaluates the four text-based measures (Figure 5.3a) and the prior-knowledge solver and attempt judge together evaluate the **limits of familiar approaches** (Figure 5.3b). In our experiments, the task judge and attempt judge use GPT 5.6 Luna, while the prior-knowledge solver pools attempts from GPT 5.6 Luna and GPT-4.1.

5.3.1 Task Judge

The task judge takes the student task, the listed prior skills, and the target concept as input without seeing any solver attempts, and outputs an outcome of *Yes*, *No*, or *Unclear* together with a brief explanation and supporting evidence for four of the five measures in Table 5.1 (Figure 5.3a).

1. **Target Alignment**

   Checks whether answering the task's central question requires the target concept.

2. **Prior-Knowledge Accessibility**

   Checks whether a student can take at least one concrete mathematical step toward the requested answer

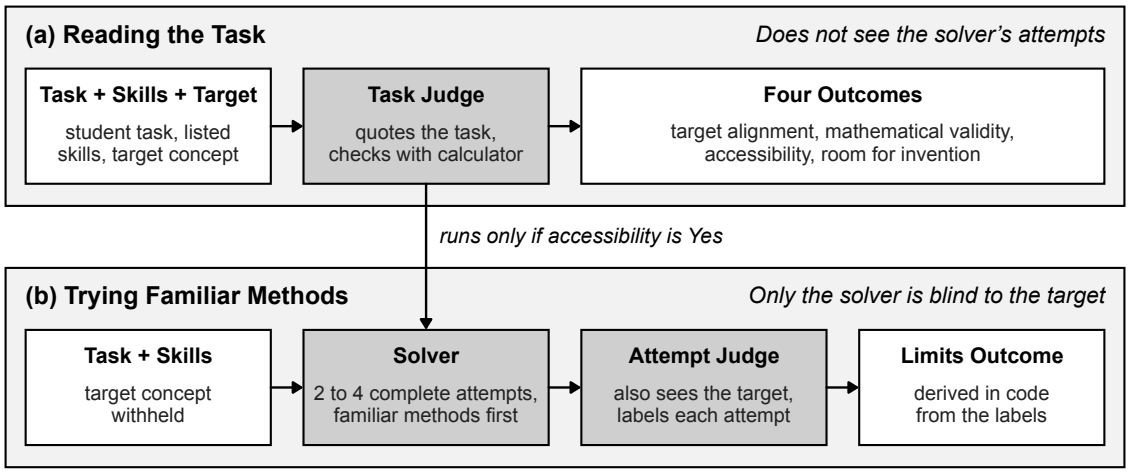


Figure 5.3: Information Separation in PF EVAL: (a) Task Judge and (b) Solver with Attempt Judge

using the listed prior skills and basic arithmetic, or whether even the first step requires an untaught concept.

3. **Room for Invention**

   Checks whether the prompt leaves the central mathematical construction open for students to build or states the target formula or step-by-step recipe in the text.

4. **Mathematical Validity**

   Checks whether the given numbers and conditions are consistent and admit a solution by verifying the numerical claims with the `SymPy` calculator from Section 4.3.2.

For the first three text-based measures, the judge must also output verbatim quotes from the student task that support its explanation, and our evaluation code verifies that every quote appears word for word in the task text so the judge cannot claim a starting step or a hint that is not written in the prompt.

5.3.2 Prior-Knowledge Solver

On tasks that pass the accessibility check, the prior-knowledge solver receives the student task and the listed prior skills while the target concept is withheld, so that it cannot steer towards the untaught formula (Figure 5.3b). It generates between two and four complete attempts that each answer every part of the task. We divide these attempts into two groups because a single familiar method and a combined strategy test different aspects of the task design.

1. **Familiar Methods**

   The first attempts apply a single listed method on its own, such as comparing only the means or only the ranges in the consistency task, and record where that method falls short.

2. **Constructed Approaches**

The remaining attempts combine several listed skills or apply general problem-solving heuristics, such as guessing and testing candidate values or organizing the givens in a table [18], to see whether a working solution can be constructed from prior knowledge.

When the listed prior skills already include a procedure that solves the task directly, the solver applies that taught procedure across all parts of the problem rather than inventing an artificially weakened baseline, ensuring that a routine textbook exercise is not mistaken for a PF task. Each attempt states the mathematical principle it relies on and any assumptions it makes, and the solver may call the calculator one arithmetic step at a time, just as a student would use a pocket calculator.

5.3.3 Attempt Judge

Because the solver is blind to the target concept, a separate attempt judge inspects each attempt against the target concept to evaluate the **limits of familiar approaches** and assigns one of the five labels shown in Figure 5.4a.

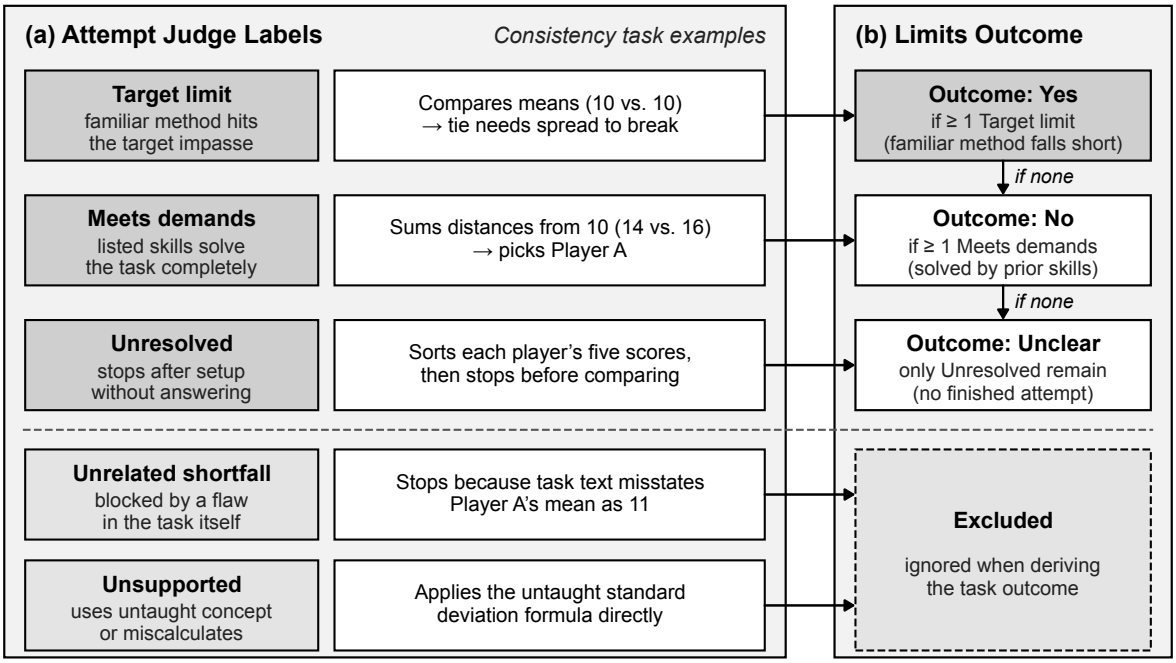


Figure 5.4: Attempt-Judge Labels (a) and Priority Rule for the Limits of Familiar Approaches (b)

Code then derives the outcome for the **limits of familiar approaches** using the priority rule in Figure 5.4b. After setting aside any *Unrelated shortfall* or *Unsupported* attempts, a single *Target limit* is enough for *Yes* even if a later constructed attempt also *Meets demands*, whereas a routine task where familiar methods reach *Meets demands* directly without any *Target limit* receives *No*.

5.4 Validating PF EVAL

PF EVAL relies on the prior-knowledge solver and the assumption that its attempts cover the familiar methods a student can try with the listed skills. If the solver overlooks a familiar method that falls short, PF EVAL would reject a working PF task, and if it misses a direct solution from prior knowledge, PF EVAL would accept a routine exercise. Beyond the solver, the two judges must also apply the five measures accurately on the finished task. Before comparing generators in Chapter 6, we therefore validate PF EVAL in two steps, testing first whether the prior-knowledge solver recovers the expected approaches on its own (Section 5.4.1) and second whether PF EVAL as a whole accepts published PF tasks and rejects flawed tasks for the right reason (Section 5.4.2).

5.4.1 Solver Recall on Expected Approaches

In PF EVAL, the prior-knowledge solver receives only the student task and the listed prior skills and generates only two to four attempts per model. We therefore need to verify whether this small sample is sufficient to surface the specific approaches a task is built around. To test this across both human-written and generated problems, we constructed a benchmark of 76 tasks with 256 expected approaches recorded in advance. The benchmark spans three task types, namely 12 published PF tasks with 28 student approaches documented in classroom studies, 12 routine and control tasks with 19 standard solutions, and 52 STRUCTPF tasks with 209 approaches planned in Stage 2a (Section 4.3.1). We run the solver exactly as in PF EVAL, pooling attempts from GPT 5.6 Luna and GPT-4.1 without tuning on this benchmark, and use a matching judge to check whether each expected approach is carried out.

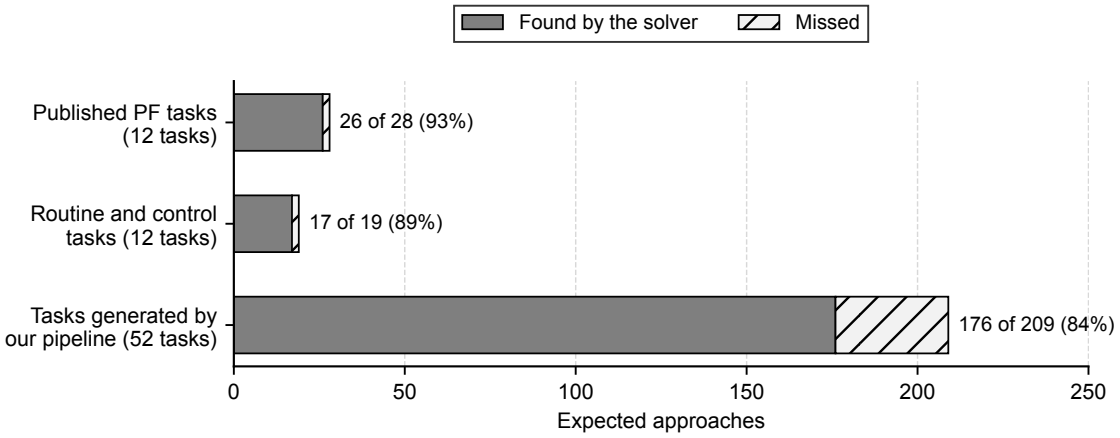


Figure 5.5: Recall of Expected Student Approaches

As shown in Figure 5.5, the solver recovers 85.5% of the expected approaches overall, with nearly all misses occurring on the pipeline-generated tasks whose planned approaches were written by the generator rather than observed in students. More importantly for task evaluation, on every PF task in the benchmark (including all published tasks), the solver tries at least one familiar method that falls short as required for a *Yes* on the limits of familiar approaches, and on every routine task it finds the direct solution without inventing artificial failures. Manual inspection of a 12-task sample also confirmed all matching decisions.

5.4.2 Validating PF EVAL on Reference Tasks

Knowing that the solver finds the expected approaches is only half of the validation. We also need to verify that the five measures and three roles work together as a complete evaluator, so that PF EVAL is neither too strict to pass real PF tasks nor too lenient to catch specific design flaws. We therefore evaluate PF EVAL on the 29 reference tasks with known expected outcomes in Table 5.2, and Figure 5.6 reports the resulting non-passing counts by group.

Table 5.2: Reference Task Groups for Validating PF EVAL

Group	Tasks	Example	Expected Outcome
Published PF tasks	12	Design a measure of consistency for three football players [19].	Yes on all five measures, with Unclear validity for one task.
Routine versions	3	Find a ship's average speed over a journey with two parts [7].	No on limits and room for invention.
One-shot tasks	5	The garden task in Figure 3.5.	No on limits, and a failing outcome for each further flaw.
Control tasks	7	Show a rectangle of 3 cm by 2 cm with an area of 5 cm^2.	No or Unclear on the measure that each task breaks.
STRUCTPF tasks	2	Invent a consistent rule for multiplying negative numbers.	Yes on all five measures for one task, No on limits for the other.

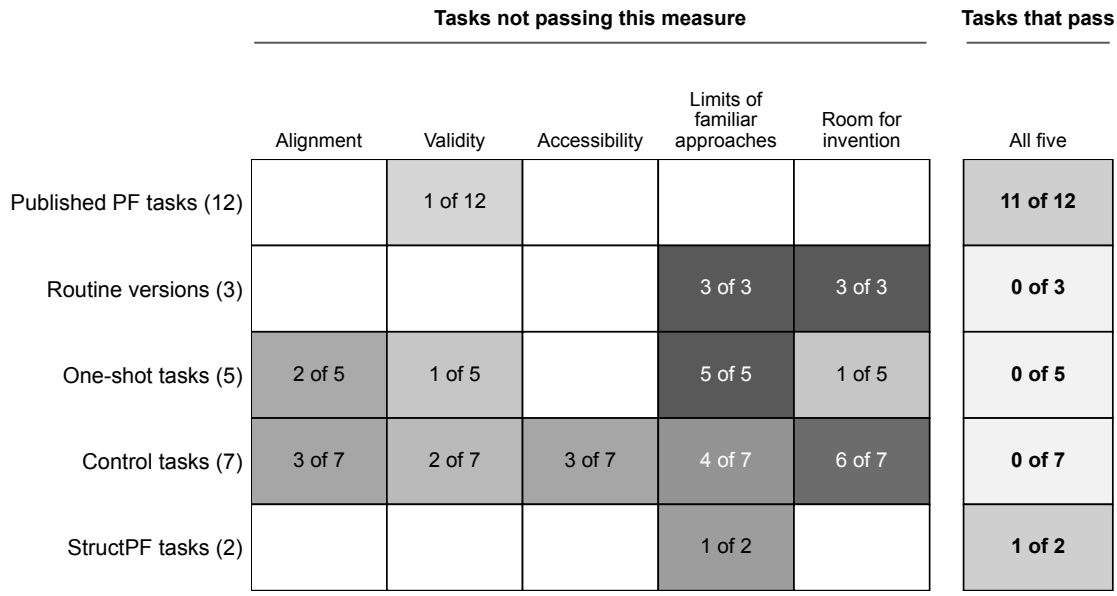


Figure 5.6: PF EVAL Outcomes on Reference Tasks by Group

The outcomes in Figure 5.6 confirm both requirements for PF EVAL to be reliable:

1. **Accepting Published PF Tasks**

   As shown in the top row of Figure 5.6, all published PF tasks pass every measure except for one *Unclear* validity outcome on the hummingbird migration task. Because that task gives each bird's wing beats

per minute only as an interval (such as 70 to 80), the resulting daily distances overlap and the winner depends on which values a student picks, so returning *Unclear* rather than *No* matches the task's actual mathematical structure.

2. **Rejecting Flawed Tasks for the Right Reason**

   Across the lower rows of Figure 5.6, none of the flawed tasks passes all five measures, and each group fails on the specific measure its flaw targets. For example, when the published consistency task is modified only by spelling out the variance rule step by step, the task judge quotes *"use exactly this rule"* to return *No* on room for invention and the solver applies that rule directly to return *No* on the limits of familiar approaches, while the one-shot garden task with a geometric contradiction (Figure 3.5) is rejected on both familiar limits and mathematical validity.

Across all 29 reference tasks, PF EVAL matches every expected outcome, accepting the published PF tasks while rejecting the routine, one-shot, and control tasks on the exact measures they violate. Having verified that the solver finds expected familiar approaches (Section 5.4.1) and that the three roles separate working PF tasks from flawed ones, we use PF EVAL in Chapter 6 to compare STRUCTPF against one-shot generation.

Chapter 6

Experiment Setup and Results

This chapter evaluates StructPF against one-shot generation across two studies to address our fourth research goal (G4, Sections 6.1 to 6.3), and then reports two negative results with simulated students that motivated grounding Stage 1 on curriculum data (Section 6.4).

6.1 Experiment Setup

We organize our main evaluation in two studies. First, Study A evaluates how well StructPF as a whole builds the underlying PF structure compared with one-shot prompting under PF Eval. Second, because knowing what students can and cannot yet do is a core bottleneck in PF design, Study B tests our Stage 1 design across 30 curriculum topics to see how well an LLM identifies the boundary of students' prior knowledge on its own and whether external curriculum grounding actually improves this bottleneck.

6.1.1 Study A: PF Eval Setup

Study A reuses the four Grade 7 settings from Chapter 3, where each setting fixes the target concept and two earlier-grade prerequisite standards in advance, as shown in Figure 6.1. We compare the 118 GPT-5 mini and 120 GPT-4.1 one-shot tasks from Chapter 3 against 120 tasks from StructPF with GPT-5 mini and Stage 1 fixed to the same prerequisites, and evaluate all 358 tasks with PF Eval in Sections 6.2.1 and 6.2.2.

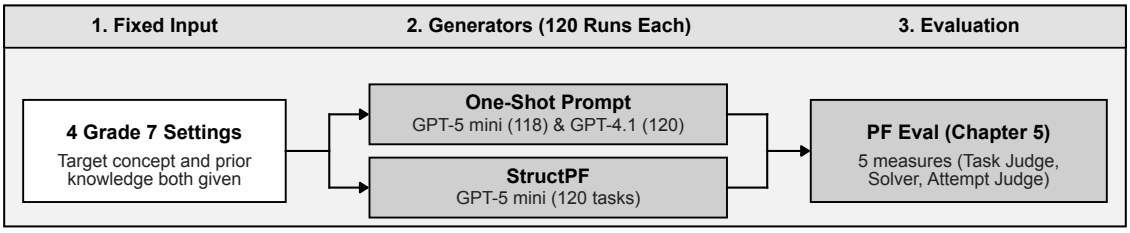


Figure 6.1: Study A Setup: Four Grade 7 Settings with Fixed Prior Knowledge

6.1.2 Study B: Curriculum Grounding Setup

While Study A provided the prerequisite standards in the setting as in Chapter 3, Study B gives only the target concept across 30 Common Core topics from Grades 5 to 10 and compares three GPT-5 mini generators with one task per topic, namely the one-shot prompt, STRUCTPF without grounding where Stage 1 relies on the model alone, and STRUCTPF with grounding where Stage 1 retrieves prerequisites from the Learning Commons graph, as shown in Figure 6.2. Comparing the one-shot prompt with STRUCTPF without grounding tests whether separating prior-knowledge identification into Stage 1 helps on its own, while comparing the two STRUCTPF variants tests how much external curriculum data adds beyond the staged structure. In each run, the generator outputs both the student task and its proposed prior skills. Because a generator might list the target formula itself as prior knowledge, we evaluate each run with the Two-Step Check in Figure 6.2 using GPT 5.6 Luna, checking first whether the proposed prior skills stay within official prerequisites in the Prior-Knowledge Check and second whether those valid skills fall short on the task numbers in the Method-Failure Check.

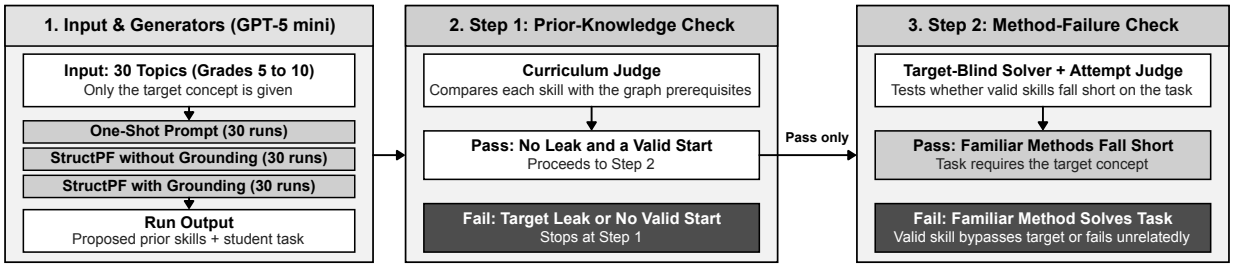


Figure 6.2: Study B Setup: Curriculum Topics and Two-Step Check

6.2 Study A: PF EVAL Experiment Results

6.2.1 One-Shot Tasks under PF EVAL

We first apply the five measures of PF EVAL to the 238 ready one-shot tasks from Study A to identify which PF requirement breaks down most often.

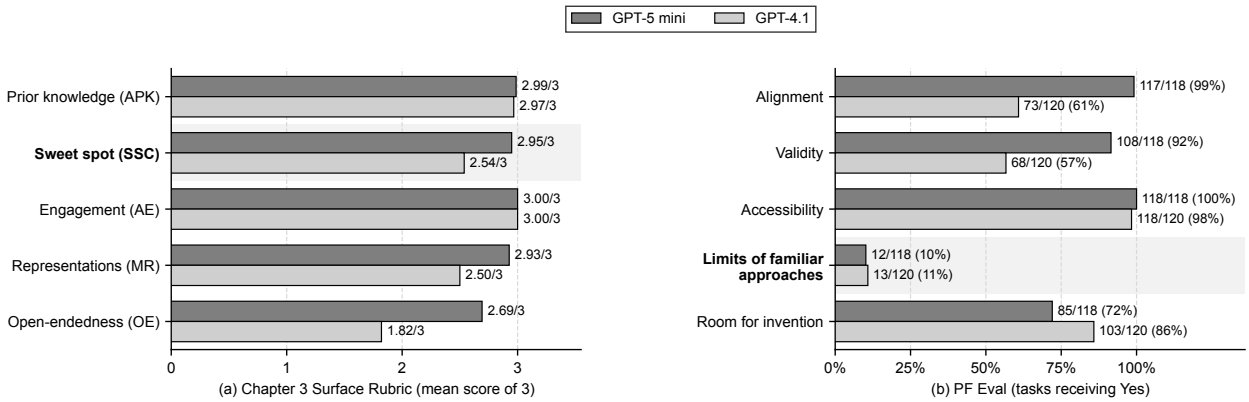


Figure 6.3: Surface Rubric versus PF EVAL on One-Shot Tasks

Figure 6.3 compares the Chapter 3 surface rubric (Figure 6.3a) directly against PF EVAL (Figure 6.3b) on the 238 one-shot tasks. Most tasks pass the soundness and accessibility checks, and on room for invention GPT-4.1 even overtakes GPT-5 mini (86% versus 72%), reversing the rubric's open-endedness scores (1.82 versus 2.69 out of 3) because GPT-5 mini's multi-part worksheets narrow the student's procedure. The sharpest discrepancy is on whether familiar methods fall short. Whereas the surface rubric rates sweet spot calibration near the ceiling at 2.95 and 2.54 out of 3, testing the tasks with the solver shows that familiar methods fall short on only 12 of 118 GPT-5 mini tasks (10%) and 13 of 120 GPT-4.1 tasks (11%). Because a single prompt writes the story and chooses the numbers at once without verifying them, it almost always picks untied values that a familiar calculation such as a single average or unit rate settles right away.

6.2.2 STRUCTPF under PF EVAL

We next evaluate the 120 tasks generated by STRUCTPF on the same four Grade 7 settings. In Table 6.1, the top rows compare STRUCTPF with the two one-shot baselines, and the bottom rows break STRUCTPF down by setting.

Table 6.1: One-Shot Prompting versus STRUCTPF on the Four Grade 7 Settings (Study A)

			Task Soundness		PF Requirements			
Generator / Setting	**Ready**	**Rubric**	**Align.**	**Valid.**	**Access.**	**Limits**	**Invention**	**All 5**
One-Shot (GPT-4.1)	120	12.8	73 (61%)	68 (57%)	118 (98%)	13 (10.8%)	103 (86%)	4 (3.3%)
One-Shot (GPT-5 mini)	118	14.6	117 (99%)	108 (92%)	118 (100%)	12 (10.2%)	85 (72%)	10 (8.5%)
STRUCTPF (GPT-5 mini)	120	12.1	**120** (100%)	**114** (95%)	117 (98%)	**96** (80.0%)	**88** (73%)	**68** (57%)
Music distributions	30	13.2	30	30	30	27 (90.0%)	28 (93%)	25 (83%)
Composite garden area	30	10.8	30	25	30	25 (83.3%)	23 (77%)	17 (57%)
Basketball ratios	30	12.3	30	29	30	23 (76.7%)	20 (67%)	15 (50%)
Shopping discounts	30	12.1	30	30	27	21 (70.0%)	17 (57%)	11 (37%)

As shown in Table 6.1, STRUCTPF increases the share of tasks where familiar methods fall short nearly eightfold over one-shot GPT-5 mini, from 12 of 118 (10.2%) to 96 of 120 (80.0%), while keeping alignment (100%), mathematical validity (95%), accessibility (98%), and room for invention (73%) high. Across the four settings, familiar methods fall short on between 70.0% of tasks in shopping discounts and 90.0% in music distributions. Taken over all five measures, 68 of the 120 STRUCTPF tasks (57%) pass every measure, compared with 10 of 118 for one-shot GPT-5 mini (8.5%) and 4 of 120 for one-shot GPT-4.1 (3.3%). By contrast, the Chapter 3 surface rubric scores STRUCTPF lower than one-shot GPT-5 mini at 12.1 versus 14.6 out of 15, because STRUCTPF withholds the rubric from Stage 3 and fixes the quantities around a single comparison rather than listing multiple representations or sub-questions.

Figure 6.4 shows two of the 68 STRUCTPF tasks that pass all five measures. In the basketball task, both drills score 20 points in 10 minutes, so the overall rate from Grade 6 cannot tell them apart, and students have to discover that a proportional relationship needs the same rate in every segment. In the shopping task, adding the two discounts gives the same price at both stores, so students have to notice that the second discount applies to the already reduced price. In both tasks, the familiar method ends in a tie, while the task still leaves the key idea to the student.

Although STRUCTPF improves the PF requirements, it still misses a measure on some of the 120 tasks.

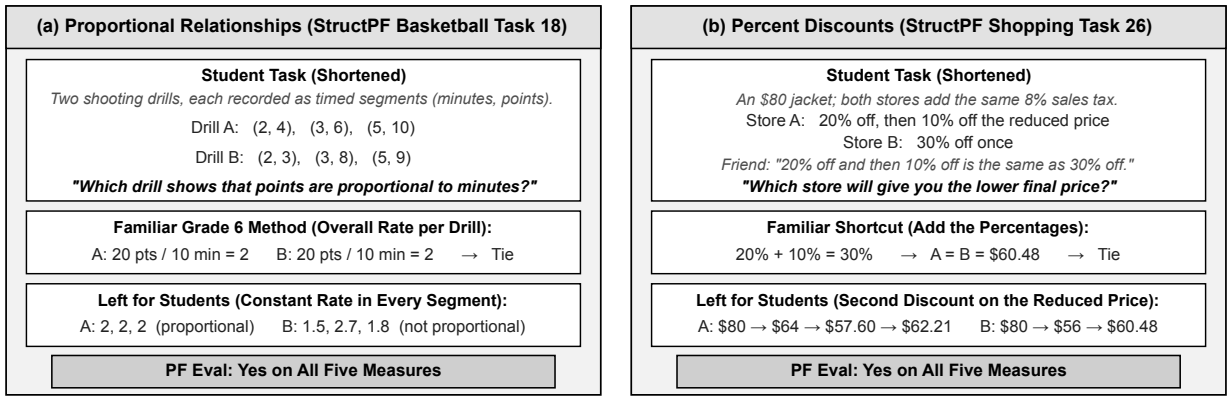


Figure 6.4: Two STRUCTPF Tasks Passing All Five PF EVAL Measures (Basketball and Shopping)

Most of these misses come from two causes.

1. **The Task Gives Away the Route (32 Tasks)**

   The task states the calculation route that students should find themselves, so room for invention fails. Shopping Task 17, for example, tells students that "the first discount reduces the current price, then the next discount applies to that reduced price", which is the key idea of successive percent change.

2. **A Familiar Method Still Solves the Task (21 Tasks)**

   On 21 of the 117 accessible tasks, the numbers do not make the simpler method fail, so a one-step calculation already reaches the right decision. In Shopping Task 10, Store A lists a jacket at $\$140$ with 30% off and 9% tax, while Store B lists it at $\$120$ with 20% off and 7% tax. After the discount alone, Store B is already cheaper ($\$96$ versus $\$98$), and its tax rate is lower as well (7% versus 9%). A student who stops after the single discount, or who simply adds the discount and tax percentages, still picks Store B, so the multistep percent calculation never changes the answer.

Neither cause comes from a missing rule in the prompts or the pipeline design. The Step 2a prompts already tell the model not to state the method in the student-facing text and to reject a design where the simplest familiar calculation already settles the question. The model simply does not always follow these instructions.

6.3 Study B: Curriculum Grounding Experiment Results

Study B evaluates whether Stage 1 curriculum grounding keeps the generator within the boundary of what students already know and leads to tasks where those familiar methods fall short. Across both steps of the Two-Step Check on the 30 topics, STRUCTPF with grounding passes on 19 topics, compared with 16 for STRUCTPF without grounding and 6 for the one-shot prompt. We examine each step in turn to see how much of this gain comes from separating Stage 1 and how much comes from grounding it on the curriculum graph.

6.3.1 Step 1: Prior-Knowledge Check

In the first step, a curriculum judge compares each proposed skill against the official prerequisite standards in the Learning Commons graph and sorts it into one of three groups, where a run passes Step 1 only if it has no target leaks and at least one valid starting skill.

- **Valid.** The skill comes before the target on the learning progression, such as one of the prerequisite standards, and students can try it on the numbers in the task.
- **Irrelevant.** The skill is mathematically wrong, belongs to an unrelated topic, or cannot be tried on the task at all.
- **Target leak.** The skill already teaches or carries out the target concept itself, for example computing a slope as $(y_2 - y_1)/(x_2 - x_1)$ when students are meant to construct the idea of slope.

A leak is the most harmful of these misjudgments, because students would arrive with the key idea already in hand and the task would leave nothing for them to invent.

Figure 6.5 counts every proposed skill across the 30 topics. In the one-shot prompt, 31 of 106 proposed skills (29%) are target leaks, which eliminate 17 of the 30 topics in Step 1. Separating prior-knowledge identification into Stage 1 in STRUCTPF without grounding already reduces target leaks to 7 of 90 skills (8%) across 5 topics, with 1 further topic failing Step 1 because its proposed skills are irrelevant, leaving 24 of 30 topics that pass Step 1. Grounding Stage 1 on the Learning Commons graph in STRUCTPF with grounding eliminates target leaks completely (0%), with 88 of 90 proposed skills valid and all 30 topics passing Step 1.

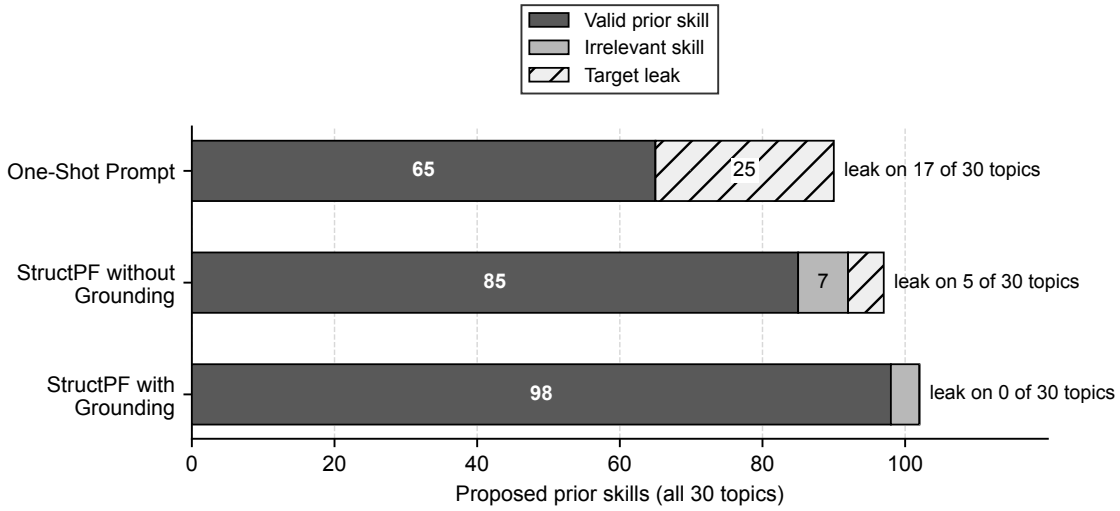


Figure 6.5: Study B Step 1: Prior-Knowledge Check by Generator

6.3.2 Step 2: Method-Failure Check

A correct judgment of prior knowledge does not guarantee that the task resists it. For the runs that pass Step 1, the target-blind solver and attempt judge of PF EVAL apply only the valid proposed skills to the task

numbers and sort each task into one of three outcomes.

- **Familiar methods fall short and need the target.** None of the valid prior skills solves the task on its own, and trying them produces a tie, conflict, or gap that requires the target concept.
- **Bypassed by a familiar method.** At least one valid prior skill already reaches the right answer on the task numbers, or the task text states the target procedure directly.
- **Falls short for an unrelated reason.** The solver gets stuck because of missing data or inconsistent numbers rather than the mathematical gap to the target concept.

Figure 6.6 shows that once the prior skills are valid, both STRUCTPF variants make those skills fall short on roughly two thirds of their eligible runs, namely 16 of 24 (67%) without grounding and 19 of 30 (63%) with grounding, compared with 6 of 13 (46%) for the one-shot prompt. Most of the remaining runs across all three generators are bypassed by a familiar method, while only a few fail for an unrelated reason.

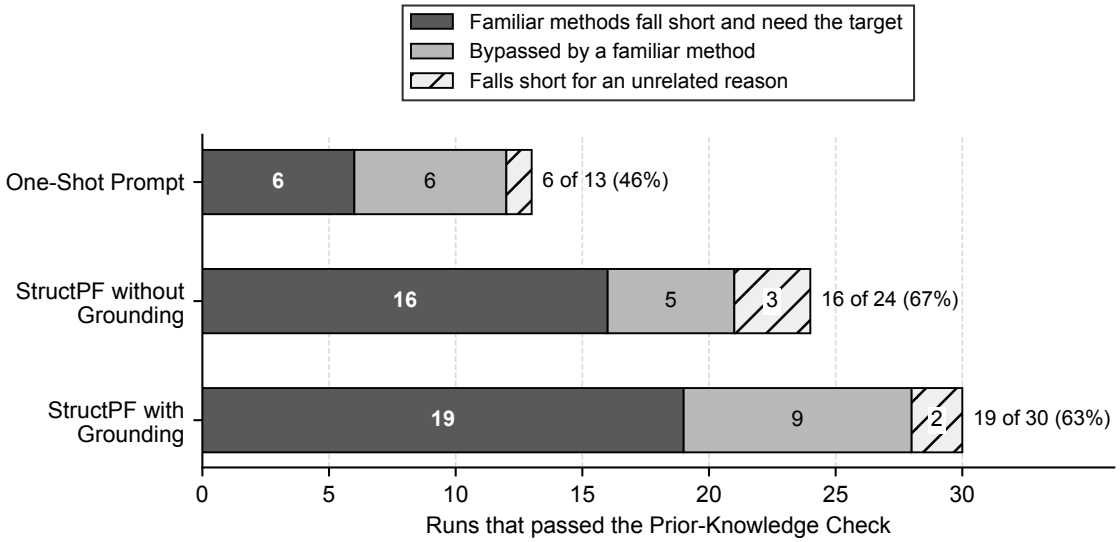


Figure 6.6: Study B Step 2: Method-Failure Check by Generator

Taken together, Figures 6.5 and 6.6 separate the contribution of each design choice in three findings.

1. **External Curriculum Grounding Is Needed to Fix the Prior-Knowledge Boundary**

   Giving prior-knowledge identification its own stage in STRUCTPF without grounding already reduces target leaks compared with one-shot prompting, because the model focuses on what students know before designing the task. However, relying on the model's own memory still leaks the target concept on several topics, and only grounding Stage 1 on the curriculum graph removes target leaks completely.

2. **Valid Prior Knowledge Alone Is Not Enough Without Staged Task Design**

   Even when the one-shot prompt identifies valid prior skills in Step 1, more than half of its tasks are still solved directly by those familiar methods in Step 2. Planning the impasse and verifying the numbers in Stage 2 are therefore still required to make valid prior skills fall short.

3. **Curriculum Grounding and Staged Design Work Together Across All Topics**

   STRUCTPF with grounding shows a slightly lower pass rate in Step 2 only because Stage 1 grounding brings all 30 topics into Step 2, including the harder topics that already failed Step 1 without grounding. Across the full set of 30 topics, combining curriculum grounding with staged design raises the number of working PF tasks from 6 for one-shot prompting and 16 without grounding to 19 with grounding.

6.4 Negative Results with Simulated Students

While developing STRUCTPF and PF EVAL, we also tested two other ways to build and evaluate Productive Failure tasks. Both attempts produced negative results, yet they are worth reporting because they revealed important pitfalls and steered our design toward the final pipeline.

6.4.1 Classroom Pathway Reproduction

Motivation

To design a task around the mathematical structure of Productive Failure, we need to know the concrete solution and error pathways that students try before instruction so that Stage 2 can choose numbers where those attempts break down. Prior work showed that simulated students given a knowledge profile can imitate student responses in several domains [15, 22], while mathematics studies warned that models often fall back on expert calculations [12, 25]. Before relying on external curriculum data, we therefore tested whether an LLM role-playing a student can think like a student and reproduce the diverse pathways observed in a real PF classroom. If it worked, we could obtain realistic student pathways directly from the model to guide PF task generation instead of grounding Stage 1 on curriculum data.

Setup

To test this against real classroom ground truth, we used Kapur and Bielaczyc's Grade 7 bicycle-handover problem [7], where the paper documents six distinct calculation strategies and error pathways produced by real student groups. We collected 24 independent attempts from simulated students across GPT-4.1 and GPT-4.1 mini under two prerequisite profiles without giving them the solution or forcing a specific error, and coded which of the six classroom pathways they actually used.

Finding

Figure 6.7 shows that 19 of the 24 simulated student attempts (79%) jumped straight to the canonical single-variable algebra equation and 21 of the 24 solved the problem completely, while 6 (25%) tried brute-force numerical guessing. None of the 24 attempts (0%) enacted the ratio-based partition or common-multiple partition error pathways that real Grade 7 groups used in the classroom, nor did any attempt draw the journey diagram documented in the paper. Because a simulated student defaults to expert algebra instead of producing the diverse student error pathways seen in real classrooms, we cannot rely on LLM-generated pathways on their own, which is why Stage 1 of STRUCTPF grounds prior methods on the real-world curriculum graph instead.

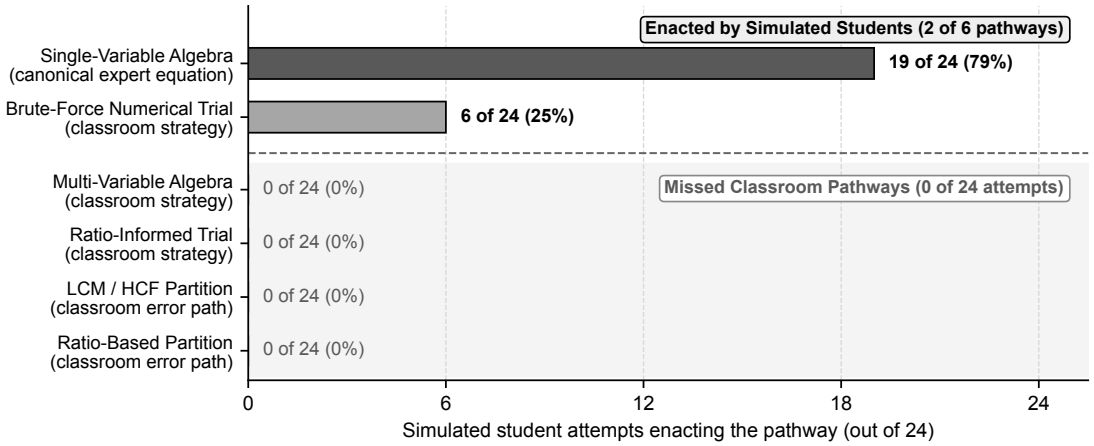


Figure 6.7: Kapur and Bielaczyc [7] Classroom Pathways Enacted by Simulated Students

6.4.2 Virtual-Classroom Feedback Loop

Motivation

Even when Stage 1 and Stage 2 use curriculum grounding, we cannot be sure how students will react to a generated task until someone attempts it. We therefore wanted to test whether a virtual classroom of simulated students could serve as an automatic refinement loop after Stage 3, where simulated student attempts are fed back into the generator to rewrite tasks that missed an intended approach.

Setup

Because any revision step might improve a draft simply by giving the model a second pass to polish its own work, we compared two matched revision paths across six generated tasks using the exact same revision prompt and call budget (Figure 6.8a). In the self-review baseline, the rewriter sees only the task text (with no simulated students), whereas in the virtual-classroom feedback loop, the rewriter receives the exact same task text plus eight simulated student attempts.

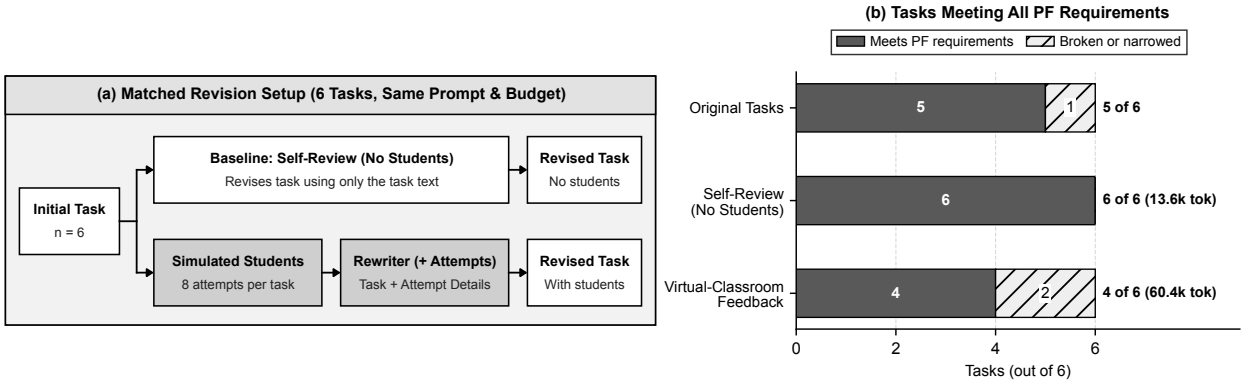


Figure 6.8: Virtual-Classroom Feedback Loop versus Self-Review

Finding

As shown in Figure 6.8b, adding the simulated student attempts actually reduced the number of valid PF

tasks from 5 of 6 to 4 of 6 while using more than four times as many tokens (60,365 versus 13,572), whereas self-review without simulated students passed all 6 of 6. Because the simulated students skipped naive shortcuts and solved the original tasks directly, their attempts made the tasks look too easy, pushing the rewriter to add extra constraints that narrowed the problem or broke its mathematical assumptions. We therefore dropped the post-Stage 3 feedback loop and evaluate tasks in PF EVAL with a calculator-checked prior-knowledge solver instead.

Chapter 7

Discussion and Conclusion

At the start of this thesis, we asked whether an LLM can generate pedagogically sound Productive Failure tasks that come close to the tasks designed by PF experts, rather than polished story problems that students can already solve with one taught procedure. Because the surface rubric from prior work could not tell these two kinds of task apart, answering this question also required a way to check the PF requirements of a generated task automatically. In this chapter, we first revisit our four research goals (Section 7.1) and then discuss what the results imply for the use of LLMs in education more broadly (Section 7.2), and the limitations of our evidence together with directions for future work (Section 7.3), before we conclude (Section 7.4).

7.1 Revisiting the Research Goals

The four goals of this thesis build on each other. G1 examined where one-shot generation and its rubric-based evaluation fall short, G2 and G3 addressed these two shortcomings with a staged generation pipeline and a requirement-based evaluation, and G4 tested whether the pipeline still works across many topics, where the generator must also identify what students already know. In the following, we summarize what Chapters 3 to 6 found for each goal and then relate the results back to the expert-designed tasks that motivated this work.

G1: One-Shot Generation and Rubric Evaluation

Our first goal was to reproduce one-shot LLM generation of PF tasks from prior work and to identify where both the generated tasks and their rubric-based evaluation fall short. In Chapter 3 and Section 6.2.1, we found that one-shot prompting writes tasks that look like Productive Failure but rarely work like it. GPT-5 mini scores 14.6 out of 15 on the rubric of Rahimi et al. [21], above the 13.38 they report for GPT-4o, yet familiar methods fall short on only 12 of its 118 tasks (10%), and GPT-4.1 does not do better with 13 of 120 (11%). The rubric misses this failure because it rates the story and wording of a task rather than testing what its numbers allow students to do.

G2: Staged Generation with StructPF

To address the weaknesses of the generated tasks, our second goal was to design StructPF, a pipeline

that generates PF tasks in smaller, checked steps instead of one pass. Study A in Chapter 6 shows that this staged design makes familiar methods fall short far more often. Because Stage 2 plans which familiar shortcut should fail and verifies the task numbers with calculations before Stage 3 writes the text, familiar methods fall short on 96 of 120 STRUCTPF tasks (80%), compared with 10% for one-shot GPT-5 mini given the same prior knowledge (Section 6.2.2). This gain does not come at the cost of task soundness, since every STRUCTPF task stays on target, 95% are mathematically valid, and room for invention stays at the one-shot level (73% versus 72%).

G3: Requirement-Based Evaluation with PF EVAL

To address the weaknesses of the rubric, our third goal was to develop PF EVAL, an evaluation that tests the three PF requirements directly, and to validate it on expert-designed and control tasks. The validation in Chapter 5 shows that testing the requirements directly separates working PF tasks from flawed ones, which the surface rubric could not do. On 29 reference tasks with known strengths and flaws, PF EVAL matches every expected outcome, accepting the published expert tasks and rejecting each routine, one-shot, and control task on the measure it breaks (Section 5.4.2). Its prior-knowledge solver also recovers 85.5% of 256 expected student approaches and, on every published PF task, tries at least one familiar method that falls short (Section 5.4.1).

G4: Comparison across Many Mathematics Topics

Our final goal was to compare STRUCTPF with one-shot generation across many mathematics topics rather than the single basketball setting tested in prior work [21]. In Study B of Chapter 6, the generator receives only a target concept from 30 Common Core topics in Grades 5 to 10 and has to identify what students already know on its own, and STRUCTPF with grounding produces the most working tasks in this setting. STRUCTPF with grounding passes both steps of the Two-Step Check on 19 topics, compared with 16 for STRUCTPF without grounding and 6 for the one-shot prompt (Section 6.3). Much of this gap arises before the task is even tested, because the one-shot prompt lists the target concept itself as prior knowledge on 17 of the 30 topics and STRUCTPF without grounding on 5, whereas grounding on the Learning Commons graph removes these target leaks entirely.

Overall Distance to Expert Tasks

Taken together, these results show that STRUCTPF closes much of the gap between one-shot generation and expert-designed PF tasks, but does not yet reach expert level. Under the same five measures of PF EVAL, 11 of the 12 published expert tasks pass every measure on their own topics, compared with 68 of 120 STRUCTPF tasks (57%) and only 10 of 118 one-shot GPT-5 mini tasks (8.5%) in Study A. Most of the remaining gap comes from tasks that give away the solution route or still let a familiar method settle the question (Section 6.2.2). Since all of these outcomes are model judgments rather than student attempts, they show how closely the generated tasks follow expert design, not whether students learn as much from them, which we return to in Section 7.3.

7.2 Implications for LLMs in Education

Although this thesis studies Productive Failure, the difficulties we encountered reflect a broader tension in using LLMs for teaching. LLMs hold vast subject knowledge, but teaching also depends on how learners

think before instruction, and this knowledge is scarce both in the training data and in the way these models generate text. We summarize four lessons that apply beyond PF task design and then describe how our approach addresses them.

Missing Knowledge about Learners

Larger models improve on what is well documented in expert writing, but not on what learners do. GPT-5 mini clearly outperforms GPT-4.1 on mathematical validity (92% versus 57%), yet familiar methods fall short on almost the same share of their tasks (10.2% versus 10.8%, Section 6.2.2). Most mathematical text records the end point of learning, while the reasoning of younger students before instruction and the links between knowledge components across grades are rarely written down. This is also why the one-shot prompt listed the target concept as prior knowledge on 17 of 30 topics, and why only curriculum grounding removed these leaks (Section 6.3). Knowledge about learners therefore has to come from curricula, teachers, and classroom data rather than from a larger model.

Bias toward the Most Common Solution

Because a language model favors probable continuations, it concentrates on the most frequent solution, while real student thinking is spread over many approaches, several of them rare. On Kapur and Bielaczyc's bicycle problem, 19 of 24 simulated attempts used single-variable algebra and none reproduced the four other pathways of real Grade 7 groups (Section 6.4.1). Recent studies likewise find that simulated students outperform real ones and abandon their assigned errors [3, 23, 25]. The same tendency affects generation, since numbers on which familiar methods fall short are an unlikely choice when a model writes a story freely, and only planning and verifying them in Stage 2 raised this share from 10% to 80%. Simulated students can therefore test what is solvable with given skills, as the solver in PF EVAL does, but they cannot stand in for a classroom.

Judges That Reward Polish

An LLM judge that reads only the task text shares the blind spots of the generator and tends to reward polish [28]. Our rubric therefore rated one-shot GPT-5 mini above STRUCTPF (14.6 versus 12.1 of 15), although familiar methods fell short on far fewer of its tasks. Whenever LLMs generate and judge teaching material, evaluation should test what the material makes a learner do, and the judge should first be validated on material with known strengths and flaws, as we did for PF EVAL.

Giving Away the Answer

LLMs are trained to be helpful and to resolve the question, whereas PF deliberately leaves the key step to the learner. Even STRUCTPF stated the solution route in 32 of 120 tasks although its prompts forbid this (Section 6.2.2). Restraint therefore cannot simply be requested in the prompt, but has to be built into the process and checked on the output, as Puech et al. [19] also had to do for an LLM tutor in PF.

Our Approach for Generation and Evaluation

These problems do not make LLMs unsuitable for pedagogical work, but they change how we should use them. Our experience with STRUCTPF and PF EVAL suggests four principles that apply equally to other teaching material, such as hints, quiz questions, or tutoring dialogues.

First, the process has to follow the pedagogical intention behind the material. STRUCTPF does not split

generation into steps for its own sake. Each stage corresponds to a decision that a PF designer makes for a specific purpose, such as choosing numbers so that a familiar method falls short, and giving this decision its own step accounts for most of the gain over one-shot generation.

Second, knowledge about how students think has to come from experts and teachers. An LLM cannot think like a student, and a larger model does not close this gap. In our pipeline, this knowledge enters through the curriculum graph, which removed all target leaks, and through published expert tasks and classroom pathways, which define the expected outcomes in PF EVAL.

Third, we have to respect the limits of LLMs in education. Although these models hold broad subject knowledge, they are distribution models that favor the most likely expert continuation rather than reasoning like a novice learner. With this limit in mind, we designed the prior-knowledge solver in PF EVAL to do its best using only the listed skills instead of relying on simulated students to role-play mistakes, and this design worked reliably across our validation tasks.

Fourth, every output has to be verified. LLMs write in a confident tone that suggests complete knowledge, yet they still make obvious mistakes. In Stage 2, for example, the model proposed a right triangle with legs 3 and 4 but a hypotenuse of 6 (Table 4.1). We therefore check every numerical condition with a calculator and verify quoted task evidence in code, rather than trusting how convincing the text reads.

7.3 Limitations and Future Work

All evidence in this thesis comes from computation and model judgments. The following four limitations bound what our results can show, and each points to a direction for future work.

7.3.1 No Evaluation with Real Students

The most important limitation is that no generated task has been tried with real students. All outcomes come from model judges and from a solver restricted to prior knowledge, so they show that a task is designed like a PF task, not that students actually invent several approaches, fail productively, or learn more in the instruction phase that follows. Simulated students cannot fill this gap, since they did not reproduce the approaches of real classrooms (Section 6.4). Future work should therefore test the generated tasks in classrooms and compare them with expert tasks on the approaches that students produce and on what they learn after instruction.

7.3.2 Few Expert-Written PF Tasks

Only 12 published expert PF tasks were available to us [19]. They serve both as the positive references for validating PF EVAL and as the expert level in our comparison, but they cover only a few topics. Section 7.1 therefore compares expert tasks on their own topics with generated tasks on other topics, and a fair comparison would require experts to write new PF tasks for the same topics as the generator. Future work should collect such topic-matched expert tasks, which would also enlarge the reference set for validating PF EVAL.

7.3.3 Subject and Curriculum Scope

Our studies cover only mathematics in English and follow the US Common Core for Grades 5 to 10 through the Learning Commons graph, with 30 tasks per setting in Study A and one task per topic in Study B. Whether the same pipeline works for other subjects, curricula, and languages remains untested. Future work could apply STRUCTPF to other subjects such as science and to other curricula by replacing the Learning Commons graph with the corresponding curriculum data.

7.3.4 Choice of Generator Model

Due to our budget, we generated all STRUCTPF tasks with GPT-5 mini rather than with larger frontier models such as GPT-5 or Claude Opus 5.5. As shown in Table 7.1, because STRUCTPF makes several calls per task, generating 120 tasks costs about $7 with GPT-5 mini, compared with roughly $35 with GPT-5 and $75 with Claude Opus 5.5 before iterative development and evaluation. Larger models might follow the design rules more reliably and reduce the remaining tasks that give away the solution route or that a familiar method still solves. However, the comparison of GPT-5 mini and GPT-4.1 in Section 7.2 suggests that a stronger model improves subject knowledge rather than knowledge of how students think, so we expect this gap to remain regardless of the model. Future work should rerun STRUCTPF with these models to test which failures disappear and which remain.

Table 7.1: Calls, Tokens, and Estimated API Cost per Task

Workflow	Model	Calls	Input	Output	Total	Cost / Task	Cost / 120
One-Shot Gen	GPT-5 mini	1.0	1.6k	2.0k	3.6k	$0.004	$0.50
One-Shot Gen	GPT-4.1	1.0	1.6k	0.3k	1.9k	$0.006	$0.70
STRUCTPF Gen	GPT-5 mini	8.0	36.0k	24.0k	60.0k	$0.060	$7.00
PF EVAL Round	GPT 5.6 Luna + GPT-4.1	4.5	19.0k	9.5k	28.5k	$0.024	$2.90
STRUCTPF Gen (est.)	GPT-5	8.0	36.0k	24.0k	60.0k	$0.290	$35.00
STRUCTPF Gen (est.)	Claude Opus 5.5	8.0	36.0k	24.0k	60.0k	$0.620	$75.00

7.4 Conclusion

In this thesis, we asked whether an LLM can generate Productive Failure tasks that come close to expert design. One-shot generation produces polished tasks that a rubric rates highly, yet familiar methods fall short on only about one in ten of them. We therefore based both generation and evaluation on how a PF expert designs and checks a task. STRUCTPF follows the decisions of a PF designer, takes prior knowledge from curriculum data, and verifies the numbers before the task is written. PF EVAL checks the PF requirements on the finished task alone, with a solver restricted to prior knowledge, and matches all 29 expected outcomes on our reference tasks. Familiar methods fall short on 80% of STRUCTPF tasks compared with 10% of one-shot tasks, and 57% compared with 8.5% pass all five measures, although STRUCTPF does not yet reach the published expert tasks and has not been tested with real students.

Beyond Productive Failure, our results show that LLMs bring broad subject knowledge and fluent writing to education, yet they remain distribution models that favor the most likely expert continuation rather than

thinking like a learner. Using them effectively in education therefore means respecting these limits, as our prior-knowledge solver does instead of role-play simulated students, supplying knowledge about learners from experts, teachers, and curricula, and verifying every output rather than trusting its confident tone. We hope that STRUCTPF and PF EVAL offer a starting point for using LLMs in this way, so that the tasks they generate let students struggle with the key idea before they are taught it.

Appendix A

Prompts and Evaluation Rubrics

A.1 Reconstructed One-Shot PF Generation Prompt

Table A.1 presents the reconstructed six-part prompt used in Chapter 3. The system prompt stays fixed across runs, while the user message supplies the target concept, theme, interest, and prerequisites for each Grade 7 setting.

Table A.1: Reconstructed One-Shot PF Prompt (Six Parts)

Component	Prompt Content
1. Role & Rules (System)	`ROLE AND OBJECTIVE`: Educational design assistant creating a mathematics PF problem for the supplied grade, target, theme, student interest, and explicitly listed prior knowledge. `RESPONSE RULES`: Write an engaging, age-appropriate narrative with the needed data and an open-ended challenge. Leave room for students to choose and explain approaches. Do not supply a worked solution, teacher commentary, or steps teaching the target concept. Output only JSON: `{"status": "ready", "title": "...", "body": "...", "question": "..."}`.
2. PF Overview (System)	`PRODUCTIVE FAILURE OVERVIEW`: Students attempt a challenging problem before direct instruction on the target concept so their initial representations and approaches support later comparison and consolidation. Support familiar knowledge while making further conceptual development useful.
3. PF Rubric (System, [21])	`PUBLISHED RUBRIC`: Use these five dimensions (1–3 anchors) as design guidance without self-scoring: **1. APK:** 1: No prerequisite concepts; 2: One prerequisite; 3: Two or more prerequisites. **2. SSC:** 1: Too easy/difficult (disengagement); 2: Moderate challenge with some failures; 3: Challenging but manageable, leading to productive engagement. **3. AE:** 1: Not relatable/meaningful; 2: Somewhat engaging, realistic; 3: Highly engaging narrative. **4. MR:** 1: Few/no methods; 2: Limited diversity; 3: Wide range of representations and methods. **5. OE:** 1: Simple, well-structured, single path; 2: Moderate complexity with limited variables; 3: Highly complex, integrating multiple variables and representations.
4. Five Literature Examples (System, STRATL [19])	`FIVE LITERATURE EXAMPLES`: Concise adaptations of human-source tasks from Puech et al. (2025) illustrating PF task forms; do not copy their topic or data when unmatched to the target. **Ex. 1 – Consistency (Kapur, 2015):** Five-season goal totals for Mike (13, 12, 15, 17, 13), Dave (12, 14, 16, 15, 13), and Ivan (14, 10, 18, 18, 15); design consistency measures using all five seasons. **Ex. 2 – Representation (Kerrigan, 2018):** Six states have populations of 1,646k; 6,936k; 154k; 2,091k; 685k; and 988k, with 250 legislative seats; propose and justify a fair seat allocation. **Ex. 3 – Land sharing (Kerrigan et al., 2021):** Four owners value four plots at Andrea (150, 300, 200, 350), Beatrice (200, 350, 100, 350), Cleo (200, 450, 200, 150), David (250 each); justify a fair division. **Ex. 4 – Shared furniture (Kerrigan et al., 2021):** Three roommates value four items at Ashley (150, 150, 175, 275), Bill (250, 200, 150, 300), Cliff (175, 125, 125, 250); justify a fair settlement. **Ex. 5 – Sharing a bicycle (Kapur, 2010):** Two friends have 5 km left and one bike (Jasmine: 150 m/min ride, 30 m/min walk; Hady: 200 m/min ride, 50 m/min walk); find where to leave it to tie.
5–6. Grade 7 Setting Specification (User, Grade 7)	Generate one paper-setting PF task as JSON for the supplied setting (target concept is untaught): **Setting 1 – Sports (Basketball):** *Target:* Recognize/represent proportional relationships. *Interest:* Basketball and Stephen Curry. *Prerequisites:* Fractional unit rates; rational addition/subtraction. **Setting 2 – Shopping (Pocket Money):** *Target:* Multistep ratio/percent problems (tax, markups, markdowns). *Interest:* Shopping and pocket money. *Prerequisites:* Proportions; rational operations. **Setting 3 – Music (Playlists):** *Target:* Compare two populations via center/variability from random samples. *Interest:* Bands, songs, playlists. *Prerequisites:* Mean, median, IQR, MAD; decimals. **Setting 4 – Home & Garden:** *Target:* Area of composite 2D figures (triangles, quadrilaterals, polygons). *Interest:* Garden, patio, or mural design. *Prerequisites:* Composing/decomposing area; rationals.

A.2 Automated Five-Dimension Rubric Judge Prompt

Table A.2 presents the prompt used for the two-pass blinded LLM rubric evaluation in Chapter 3. All five dimension names and 1–3 scoring anchors are retained verbatim from Table 1 of Rahimi et al. [21], while the input specification supplies the shared Grade 7 setting and student-facing task text without revealing the generating model.

Table A.2: Automated Five-Dimension Judge Prompt (Adapted from Rahimi et al. [21])

Component	Prompt Content
1. Evaluator Role (System)	Evaluate a mathematics Productive Failure problem using the five dimensions in Rahimi et al. (2025), Table 1. The scoring anchors below are the paper's verbatim anchors. This is an automated rating, not a human rating or observed student learning outcome.
2. Verbatim Rubric Anchors (System, [21])	**1. Activation of Prior Knowledge (APK):** 1 (Low): No prerequisite concepts; 2 (Moderate): Includes one prerequisite concept; 3 (High): Includes two or more prerequisite concepts. **2. Sweet Spot Calibration (SSC):** 1 (Low): Too easy or too difficult: disengagement; 2 (Moderate): Moderate challenge with some failures; 3 (High): Well-calibrated: challenging but manageable, leading to productive engagement. **3. Affective Engagement (AE):** 1 (Low): Not relatable or meaningful: fails to engage; 2 (Moderate): Somewhat engaging with relatable/realistic elements; 3 (High): Highly engaging, presented in meaningful narrative or real-world scenario. **4. Multiple Representations & Solutions (MR):** 1 (Low): Few or no representations or solution methods; 2 (Moderate): Some diversity but limited; 3 (High): Wide range of representations and solution methods; fosters creativity & collaboration. **5. Open-endedness / Ill-structuredness (OE):** 1 (Low): Simple, well-structured, single path; 2 (Moderate): Moderate complexity with limited variables; 3 (High): Highly complex, requiring integration of multiple variables and representations.
3. Input Scope & Reference Rules (System)	`INPUT: task_setting` states the intended PF exploration use, the untaught learning target, and the assigned theme and student interest. `learner_context` lists the assumed prior knowledge. `student_task` is the exact text students receive and is the only item rated. Use `task_setting` and `learner_context` only as the reference a trained rater would know: prerequisites for APK are the listed prior knowledge, challenge for SSC is for students who know that prior knowledge but not the target, and relatability for AE is for students with the stated interest. Do not reward a problem for restating the setting, and do not rate the setting itself.
4. Rating & Output Contract (System)	Apply each dimension independently and give brief, concrete evidence from the supplied problem and learner context before each score. Do not target a score distribution, match published model averages, or add extra scoring rules (such as requiring named characters, deliberately wrong answers, fixed counts of display formats, or banning a unique numerical answer). Assess what the problem supports rather than merely what it claims. If evidence is insufficient, return `null` for its score and explain why; keep mathematical concerns in a separate list without inventing a sixth score. Return only JSON with keys `APK`, `SSC`, `AE`, `MR`, `OE` (each `{"evidence": "...", "score": 1|2|3|null}`), and `mathematical_concerns` (`[...]`). Code computes the 5–15 total.
5. Blinded Task Payload (User)	JSON object containing `task_setting` (`intended_use`, `learning_target`, `theme`, `student_interest`), `learner_context` (`grade`, `subject`, `jurisdiction`, `assumed_prior_knowledge`), and `student_task` (concatenated student-facing `title`, `body`, and `question`), shuffled under a blind task ID with all model and condition labels removed.

A.3 STRUCTPF Prompt Structure

Tables A.3 to A.8 summarize the STRUCTPF prompts for Steps 2a and 2b of Stage 2 and for Stage 3, one table per prompt. Each prompt has a system message with the rules and output format, and a user message with the case-specific inputs. The full text is in `src/generation/prompts/` (`design_briefs.yaml` and `design_data.yaml`).

Table A.3: Step 2a Brief Proposal Prompt (`PROPOSE_BRIEF`)

Field	Summary
Purpose	Propose one internal design brief that gives students a challenge without stating the target concept.
Key rules	Name the intended discovery and one plausible first answer before writing the student-facing goal. Write the goal as a decision or construction students must make, not as a demonstration of the key idea. Keep the needed data visible and do not announce the target conclusion. Describe at least one defensible route to the goal.
Inputs	`target_concept`, `topic`, `grade`, `subject`, `prior_context`; `rubric_guidance` and `proposal_feedback` (system)
Output	JSON brief with `goal`, `key_idea`, `naive_answer`, `situation_and_rules`, `student_question`, `approaches`, `successful_route`, `limitation`, and `number_requirements`

Table A.4: Step 2a Brief Selection Prompt (`SELECT_BRIEF`)

Field	Summary
Purpose	Choose the most promising candidate brief for these learners, or name the concern that most needs revision.
Key rules	A successful familiar route is not a rejection reason. Flag a candidate whose student-facing fields announce the discovery, prescribe the target claim, or presuppose it. Do not invent missing assumptions to rescue a candidate. If every candidate discloses the idea, choose the best and name the specific revision needed.
Inputs	`candidates`, `target_concept`, `grade`, `prior_context`; `rubric_guidance` (system)
Output	Selected candidate, reasons, and `main_concern`

Table A.5: Step 2a Quantity Translation Prompt (`TRANSLATE_BRIEF`)

Field	Summary
Purpose	Encode the selected brief as technical quantities and checkable conditions, without redesigning it.
Key rules	Preserve the whole goal, assumptions, and allowed actions. Do not shrink an allocation to one recipient or replace an invention goal with a prescribed formula. Declare the given inputs and derived quantities that the relationships need. Instantiate one concrete case with fixed positive list lengths when counts are open.
Inputs	`design_brief`, `selection`, `target_concept`, `grade`, `prior_context`, `translation_feedback`; `quantity_contract` and `checker_contract` (system)
Output	JSON quantity objects and an optional `prior_contrast` intent, with `status` and `reason`

Table A.6: Step 2b Number Proposal Prompt (`PROPOSE_DATA`)

Field	Summary
Purpose	Choose concrete numbers, and calculator conditions that check them, for the selected design.
Key rules	Choose numbers only; preserve the goal, relationships, allowed actions, and question. Link each essential requirement to a condition with an executable calculation and predicate. Use lowercase quantity names and finite numeric values.
Inputs	`design_brief`, `requirements`, `student_context`, `prior_context`, `feedback`
Output	JSON with `quantities`, `values`, and `conditions` (statement, requirement indices, calculation, reasoning)

Table A.7: Step 2b Number Repair Prompt (`REPAIR_DATA`)

Field	Summary
Purpose	Adjust only the numerical values after the calculator reports exact results.
Key rules	Keep names, units, list sizes, design, formulas, and predicates unchanged. Do not remove a failed condition or change the successful route to pass a check. Return actual numbers, not formulas.
Inputs	`design_brief`, `proposal` (frozen), `feedback` (exact results)
Output	JSON `values` object containing every original name

Table A.8: Stage 3 Task Writing Prompt (`WRITE_DATA_TASK`)

Field	Summary
Purpose	Write the student-facing task from the verified setup and checked data.
Key rules	Keep the situation, units, and rules needed to define the problem. Withhold internal design notes and solutions, and record each withheld item with its source and quote. Always return a complete task; unresolved concerns go to `blocking_issues`, not an empty task. A ready status means a draft was produced, not that it is validated.
Inputs	`student_context`, `prior_context`, `student_draft`, `do_not_state`, `check`, `proposal`
Output	JSON with `status`, `blocking_issues`, `title`, `body`, `question`, and a `withheld` list (kind, source, quote, reason)

Bibliography

[1] Elaf Alhazmi, Quan Z. Sheng, Wei Emma Zhang, Munazza Zaib, and Ahoud Alhazmi. Distractor generation in multiple-choice tasks: A survey of methods, datasets, and evaluation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[2] Jonathan Cook, Tim Rocktaschel, Jakob Foerster, Dennis Aumiller, and Alex Wang. TICKing all the boxes: Generated checklists improve LLM evaluation and generation. *arXiv preprint arXiv:2410.03608*, 2024.

[3] Heejin Do, Shashank Sonkar, and Mrinmaya Sachan. Simulating students or sycophantic problem solving? on misconception faithfulness of LLM simulators. *arXiv preprint arXiv:2605.12748*, 2026.

[4] Wanyong Feng, Jaewook Lee, Hunter McNichols, Alexander Scarlatos, Digory Smith, Simon Woodhead, Nancy Otero Ornelas, and Andrew Lan. Exploring automated distractor generation for math multiple-choice questions via large language models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024.

[5] Manu Kapur. Productive failure. *Cognition and Instruction*, 26(3):379–424, 2008.

[6] Manu Kapur. Productive failure in learning math. *Cognitive Science*, 38(5):1008–1022, 2014.

[7] Manu Kapur and Katerine Bielaczyc. Designing for productive failure. *Journal of the Learning Sciences*, 21(1):45–83, 2012.

[8] Manu Kapur and Jae Hyun Lee. Designing for productive failure in mathematical problem solving. *Cognitive Science*, 2009.

[9] Manu Kapur, Janan Saba, and Ido Roll. Prior math achievement and inventive production predict learning from productive failure. *npj Science of Learning*, 8:15, 2023.

[10] Learning Commons. Learning commons knowledge graph (version 1.14.0). https://github.com/learning-commons-org/knowledge-graph, 2025. CC BY 4.0 export integrating Student Achievement Partners' Coherence Map and Achievement Network learning components.

[11] Yukyung Lee, Joonghoon Kim, Jaehee Kim, Hyowon Cho, Jaewook Kang, Pilsung Kang, and Najoung Kim. CheckEval: A reliable LLM-as-a-judge framework for evaluating text generation using checklists. *arXiv preprint arXiv:2403.18771*, 2025.

[12] Naiming Liu, Shashank Sonkar, Zichao Wang, Simon Woodhead, and Richard G. Baraniuk. Novice learner and expert tutor: Evaluating math reasoning abilities of large language models with misconceptions. *arXiv preprint arXiv:2310.02439*, 2023.

[13] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[14] Katharina Loibl and Nikol Rummel. Knowing what you don't know makes failure productive. *Learning and Instruction*, 34:74–85, 2014.

[15] Xinyi Lu and Xu Wang. Generative students: Using LLM-simulated student profiles to support question item evaluation. In *Proceedings of the Eleventh ACM Conference on Learning @ Scale*, pages 16–27, 2024.

[16] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. SymPy: symbolic computing in Python. *PeerJ Computer Science*, 3:e103, 2017.

[17] Steven Moore, Huy A. Nguyen, Tianying Chen, and John Stamper. Assessing the quality of multiple-choice questions using GPT-4 and rule-based methods. In *European Conference on Technology Enhanced Learning (EC-TEL)*, 2023.

[18] George Pólya. *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, Princeton, NJ, 1945.

[19] Romain Puech, Jakub Macina, Julia Chatain, Mrinmaya Sachan, and Manu Kapur. Towards the pedagogical steering of large language models for tutoring: A case study with modeling productive failure, 2025. arXiv:2410.03781, revised March 18, 2025.

[20] Seyedahmad Rahimi, Deniz Ercan, Ran Gao, Salah Esmaeiligoujar, Maryam Babaee, Hongming Li, Shan Zhang, Ben Seiyon Lee, Avery H. Closser, and Anthony F. Botelho. ProductiveMath: A generative-AI-powered app to support productive failure teaching. In *Artificial Intelligence in Education. Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium, Blue Sky, and WideAIED*, Communications in Computer and Information Science, pages 344–351, 2025.

[21] Seyedahmad Rahimi, Salah Esmaeiligoujar, Deniz Ercan, Maryam Babaee, and Ran Gao. Small vs. large language models: Comparing language models in generating productive failure math problems. In *Proceedings of the International Conference of the Learning Sciences (ICLS)*, pages 1223–1227, 2026.

[22] Alexander Scarlatos, Nigel Fernandez, Christopher Ormerod, Susan Lottridge, and Andrew Lan. SMART: Simulated students aligned with item response theory for question difficulty prediction. In

Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 25071–25094, 2025.

[23] Alexander Scarlatos, Jaewook Lee, Simon Woodhead, and Andrew Lan. Simulated students in tutoring dialogues: Substance or illusion? *arXiv preprint arXiv:2601.04025*, 2026.

[24] Tanmay Sinha and Manu Kapur. When problem solving followed by instruction works: Evidence for productive failure. *Review of Educational Research*, 91(5):761–798, 2021.

[25] KV Aditya Srivatsa, Kaushal Kumar Maurya, and Ekaterina Kochmar. Can LLMs reliably simulate real students' abilities in mathematics and reading comprehension? *arXiv preprint arXiv:2507.08232*, 2025.

[26] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, and Zhifang Sui. Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 9440–9450, 2024.

[27] Zichao Wang, Angus Lamb, Evgeny Saveliev, Pashmina Cameron, Yordan Zaykov, José Miguel Hernández-Lobato, Richard E. Turner, Richard G. Baraniuk, Craig Barton, Simon Peyton Jones, Simon Woodhead, and Cheng Zhang. Results and insights from diagnostic questions: The NeurIPS 2020 education challenge. In *Proceedings of the NeurIPS 2020 Competition and Demonstration Track, PMLR*, volume 133, 2021.

[28] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.